Operations Research with QSB

An Academic and Practical Manual

Using the application, modeling problems, and interpreting results

QSB Project

Editors: the QSB academic and development community

Verified working edition
October 11, 2026

Editorial note

This manual accompanies the QSB macOS application and command-line tool. It presents operations research fundamentals, original reproducible examples, and instructions for formulating, validating, solving, and interpreting models. Its coverage matches the project's documented state on October 11, 2026. The roadmap remains the public source of truth for software status.

The examples in `manual/examples/` were created for this project. They do not require WinQSB files or reproduce its manual. The compatibility sections explain how to import your own files without redistributing proprietary material.

How to read this manual

Start with the application guide and the linear programming example. Then turn to the chapter for the model family you need. Each chapter separates model assumptions, the procedure in QSB, results, and method limitations. A numerical solution does not by itself certify that the formulation accurately represents the real problem.

Run commands from the repository root. JSON numbers use a decimal point; this English edition also uses decimal points in mathematical text. Results are rounded for readability. Verification compares numbers using tolerances rather than textual equality of floating-point values.

Reproducibility

Input files are part of the manual's sources. The verifier runs the associated commands and checks the expected results. The manual distinguishes native educational solvers from optional external engines, and does not attribute guarantees of optimality to heuristics or local methods.

Both language editions share the same JSON examples and bibliography. Names stored in those examples may be Spanish, such as `Lanzar` or `Operativa`. They are retained as exact identifiers in this edition; English explanations do not rename the underlying data. Application labels, commands, JSON keys, units, numerical values, and solver guarantees retain the same meaning in both editions.

Contents

Editorial note **ii**

I Using QSB and modeling fundamentals **1**

1 Practical application guide **2**
1.1 A reproducible first walkthrough . . . 2
1.2 Creating, importing, and editing . . . 2
1.3 Validating, running, and interpreting states . . . 3
1.4 Saving models and results . . . 3
1.5 Keyboard controls and alternatives to charts . . . 3
1.6 The terminal and normalized JSON . . . 4
1.7 Troubleshooting . . . 5

2 Methodological foundations **6**
2.1 Purpose of the manual . . . 6
2.2 The modeling cycle . . . 6
2.3 Validity and exactness . . . 6
2.4 Computational traceability . . . 7
2.5 Comparison and sensitivity . . . 7

II Optimization and decision making **8**

3 Linear programming **9**
3.1 What a linear model represents . . . 9
3.2 Example: product mix . . . 9
3.2.1 Reproducible input and execution . . . 10
3.2.2 Result and interpretation . . . 10
3.3 Duality and sensitivity: what can be inferred . . . 10
3.4 Method, backends, and limitations . . . 11
3.5 Exercises . . . 11

4 Integer and mixed-integer programming **12**
4.1 Indivisible decisions . . . 12
4.2 Example: minimum coverage with integer activities . . . 12
4.2.1 CLI and macOS execution . . . 12
4.2.2 Optimum and verification . . . 13
4.3 Branch and bound . . . 13
4.4 Interpretation and common mistakes . . . 14
4.5 Exercises . . . 14

5 Goal, quadratic, and nonlinear programming 15
5.1 Goal programming . . . 15
5.1.1 Priorities and deviations . . . 15
5.1.2 Example: protecting a production commitment . . . 15
5.1.3 macOS workflow and solver character . . . 16
5.2 Quadratic programming . . . 16
5.2.1 Objective convention and assumptions . . . 16
5.2.2 Example: adjustment with limited capacity . . . 17
5.2.3 macOS workflow and limitations . . . 17
5.3 Nonlinear programming . . . 18
5.3.1 Expressions and domain . . . 18
5.3.2 Example: constrained distance . . . 18
5.3.3 macOS use and approximate guarantee . . . 19
5.4 How to compare and review results . . . 19
5.5 Exercises . . . 19

6 Network models 21
6.1 From a situation to a graph . . . 21
6.2 Shortest path and minimum spanning tree . . . 21
6.3 Maximum flow . . . 22
6.4 Traveling salesperson . . . 22
6.5 Assignment and transportation . . . 22
6.6 Minimum-cost flow . . . 23
6.7 macOS use and backend selection . . . 23
6.8 Exercises . . . 23

7 Decisions, games, and dynamic programming 25
7.1 Choosing under uncertainty . . . 25
7.1.1 Payoff tables and information . . . 25
7.1.2 Bayesian updating . . . 26
7.1.3 Trees and backward induction . . . 26
7.2 Zero-sum games . . . 27
7.3 Dynamic programming: state, stage, and action . . . 27
7.3.1 Bounded knapsack . . . 28
7.3.2 Stage-by-stage routing (Stagecoach) . . . 28
7.3.3 Production and inventory . . . 28
7.4 Use, metadata, and shared limits . . . 29
7.5 Exercises and checks . . . 29

III Operations planning 30

8 Inventory models 31
8.1 The decision being modeled . . . 31
8.2 EOQ and gradual production . . . 31
8.2.1 Example: replenishing notebooks . . . 31
8.3 All-units quantity discounts . . . 32
8.4 The newsvendor model . . . 32
8.5 Lot sizing . . . 32
8.6 Review with uncertain demand . . . 33
8.7 macOS workflow and traceability . . . 33

8.8 Exercises . . . 33

9 Scheduling and sequencing **34**
9.1 Problem and assumptions . . . 34
9.2 Flow shop: a common route . . . 34
9.3 Job shop: individual routes . . . 34
9.4 Reading the Gantt chart and using macOS . . . 35
9.5 Exercises . . . 35

10 Projects and production planning **36**
10.1 CPM: precedence and the critical path . . . 36
10.2 PERT: expected duration and uncertainty . . . 37
10.2.1 CPM and PERT in macOS . . . 37
10.3 Aggregate planning . . . 37
10.4 MRP: requirements explosion . . . 38
10.5 Exercises . . . 39

11 Facilities and line balancing **40**
11.1 Locating a facility . . . 40
11.2 Balancing a line . . . 40
11.3 Facility layout . . . 41
11.4 Backend, validation, and decision . . . 42
11.5 Exercises . . . 42

IV Uncertainty and quality **43**

12 Queuing models **44**
12.1 From flow to waiting . . . 44
12.2 M/M/1 in steady state . . . 44
12.2.1 Example: a service counter . . . 44
12.3 Multiple servers and finite capacity . . . 44
12.3.1 Example: two counters and three waiting places . . . 45
12.4 Costs, interface, and limitations . . . 45
12.5 Exercises . . . 46

13 Forecasting and regression **47**
13.1 A forecast is an operational hypothesis . . . 47
13.2 Data and errors . . . 47
13.3 Linear trend: a complete example . . . 47
13.4 Moving average and smoothing . . . 48
13.5 Multiplicative seasonality . . . 48
13.6 Explanatory regression . . . 48
13.7 Working in macOS and evaluation . . . 49
13.8 Exercises . . . 49

14 Markov processes and simulation **50**
14.1 Two ways to describe uncertainty . . . 50
14.2 Finite chains and the row convention . . . 50
14.2.1 Example: machine availability . . . 50
14.3 Discrete-event simulation . . . 51
14.3.1 Example: a simulated service counter . . . 51

14.4 macOS workflow 52
14.5 Exercises 52

15 Quality control and acceptance sampling **53**
15.1 Monitoring a process or accepting a lot 53
15.2 c and p charts 53
15.3 Mean and range charts 54
15.4 Pareto and normal probability 54
15.5 Signals and the quality interface 54
15.6 Single acceptance plan 55
15.7 Double plans and design 55
15.8 Exercises 56

A Terminology reference **57**

List of Tables

3.1 Product mix data 9
3.2 Linear optimum verified with QSB 10

5.1 Lexicographic result for the production case 16

Part I

Using QSB and modeling fundamentals

Chapter 1

Practical application guide

1.1 A reproducible first walkthrough

QSB provides a macOS application and a command-line tool called `qsb`. Both share the models, validators, and solvers in `QSBCore`. The current interface uses English labels; this manual retains those labels so they can be located on screen.

To begin, open `manual/examples/lp_sample.json` using **Model > Open Model...**. The model maximizes $50X_1 + 60X_2$ subject to $2X_1 + 3X_2 \leq 180$, $3X_1 + 2X_2 \leq 150$, and nonnegative variables.

1. In **Definition**, check names, coefficients, relations, bounds, and variable types. The variables here are continuous.

2. Click **Validate** and review **Validation**. A valid model is a consistent input; it is not yet a solution.

3. In **Run**, select **QSB Native** and click the **Run** button; you can also use **Solve** in the toolbar or **Solve > Run Current Model** in the menu.

4. In **Summary**, check $X_1 = 18$, $X_2 = 48$, and objective 3780. Both constraints hold with equality. Also review the solution status and metadata before interpreting the number as an optimum.

5. Export the model and solution separately. Reopen the exported model to preserve a reproducible input.

The **Samples** menu provides built-in examples, but these may differ from the manual's files. To reproduce a cited result, use the file specified in its chapter.

1.2 Creating, importing, and editing

New Model... opens the family selector and creates an initial model. **Open Model...** accepts normalized JSON and recognized legacy formats. The family determines which controls appear. Native editors let you work with names, tables, and typed fields; other variants use a structured form and JSON as an interchange surface.

The **Definition** panel defines the problem; **Overview** summarizes its family and state. **JSON** lets you inspect its representation. When **Apply JSON to ... Editor** is available, explicitly apply the edited text to replace the native draft. Writing JSON does not mean the changes have already been incorporated into the table. Review the diagnostics if applying the JSON fails and preserve the text you need to repair.

Some tables offer **Copy Table** and **Paste Table** to exchange ranges with spreadsheets. Check headers, column order, separators, units, and names after pasting. Malformed numeric fields are not silently converted to zeros. In editors that flag incomplete imported dimensions, review the cells and shown values before accepting **Use Displayed Dimensions**.

1.3 Validating, running, and interpreting states

The application distinguishes empty, editing, validating, invalid, and valid model states. Execution has separate states: **Not run**, **Validating**, **Solving**, **Solved**, and **Failed**. Read a Solved status alongside the mathematical condition reported by the family: a method may return an approximation, local solution, or heuristic policy rather than a global optimum.

Validation displays model errors and warnings. Correct errors before solving; a warning requires judgment even when it allows you to proceed. **Run Details** lets you review the backend and execution information. When a long run displays progress, wait for completion before repeating Run. There is no universal cancellation control; some editors' asynchronous paths prevent an old response from replacing a new draft. This does not cancel the background computation.

QSB Native Deterministic educational methods. Exactness and limits depend on the family and are explained in the corresponding chapters.

Validate only Checks the input and produces diagnostics; it does not compute decisions or optimal values.

External solver Integrations available only for certain families. Availability depends on configuration and the installed solver. The presence of this option does not imply universal external support.

For LP/ILP and QP, consult the HiGHS conditions in the optimization chapter. Other families may report that the external backend is unavailable. Switch to QSB Native or Validate only according to the work you want to do.

1.4 Saving models and results

The toolbar's **Export** button prepares JSON from the current draft. If a native field cannot be converted into the model, QSB displays validation rather than exporting old data. The **Model > Export Model JSON...** menu opens export of the model text; to ensure recent fields are synchronized, first use Validate or the toolbar's Export button. **Export Solution JSON...** saves the existing result and is enabled when solution content is present.

These are distinct documents: the model describes the problem to solve, and the solution records what a run returned. Output formats vary by family; some include the model and others do not. Always preserve both, together with the command, backend, software version, and study assumptions. Do not use a solution document as a model input.

1.5 Keyboard controls and alternatives to charts

The application defines the following shortcuts:

Action	Keys
New Model	Cmd+N
Open Model	Cmd+O
Export Model JSON	Cmd+S
Export Solution JSON	Cmd+Shift+S
Validate Model	Cmd+Shift+V
Run Current Model	Cmd+R
Solve ILP	Cmd+I

Buttons are enabled according to family and state. Solve ILP and Solve LP Relaxation are specific to linear/integer programming; the relaxation does not enforce integrality. Some editors use Cmd+Shift+N to add rows. Use Tab to move through native controls and the menus to locate actions; focus behavior depends on the editor.

Tables and JSON let you inspect data without relying on chart colors. Native results include accessible labels and values in several families, but this alone does not establish a complete VoiceOver review. Preserve the numerical tables for reproducible analysis. The macOS window has a minimum size of 980 by 620 points; use scrollable areas and close the Inspector when you need more width.

1.6 The terminal and normalized JSON

From the repository root, SwiftPM builds and runs the tool:

```
swift run qsb
swift run qsb validate manual/examples/lp_sample.json
swift run qsb solve-json manual/examples/lp_sample.json --backend native
```

Without arguments, `qsb` displays the command catalog. After `swift build`, you can run `.build/debug/qsb` directly. To save output and separate errors:

```
.build/debug/qsb solve-json manual/examples/lp_sample.json \
  --backend native > /tmp/qsb-lp-solution.json 2> /tmp/qsb-lp-errors.txt
```

JSON uses decimal points, double-quoted strings, and `null` for optional fields that allow it. It does not include comments. Each family has its own schema: LP uses a direct object, while many other families use a wrapper with `kind` and `model`. Copy the structure of the family's example, keeping names, vectors, and matrices aligned.

`solve-json` detects the family of a normalized input. `validate` accepts normalized or legacy inputs. `solve` targets legacy models; do not interchange commands indiscriminately. The chapters provide family-specific commands, such as `solve-dp-json` or `solve-decision-json`. `validate-json` is specific to LP/ILP, rather than a universal validator.

To import your own legacy input, the general procedure is:

```
qsb import-legacy-json /ruta/al/modelo > /tmp/qsb-model.json
qsb validate /tmp/qsb-model.json
qsb solve-json /tmp/qsb-model.json --backend native
```

Here `qsb` assumes that the executable is on PATH; you can replace it with `swift run qsb`. The original files must be locally accessible and used in accordance with their permissions. The manual's examples are independent of proprietary material.

1.7 Troubleshooting

Unreadable JSON Check syntax, key names, and types; a decoding error may occur before structured diagnostics can be generated.

Invalid model Consult `isValid`, severity, code, message, and diagnostic path when included in the output. Correct the input and validate again.

No feasible solution The structure may be valid even though constraints or demand cannot be satisfied. Review bounds and balances.

Unavailable backend Check which families support it and its configuration. Validate only does not provide an alternative solver result.

Unexpected result Review units, objective sign, state order, constraints, options, and exactness. Compare with simple calculations before changing algorithms.

The CLI exits successfully when it completes a command and unsuccessfully when an error is thrown, including incorrect usage. Some validations return a report with `isValid: false` normally; the exit code alone does not prove that the model is valid. An automated workflow must also inspect the validation document and result status. The absence of syntax errors does not demonstrate that the model accurately represents the real system.

Chapter 2

Methodological foundations

2.1 Purpose of the manual

Operations research builds and analyzes quantitative models to support decisions under constraints. This manual combines foundations with practical use: each example turns a problem statement into QSB data and explains what the result means. The problems are original teaching examples, with normalized inputs available in `manual/examples/`; they do not require proprietary manuals or files from earlier programs.

You can start with the application guide and a linear programming example, or go directly to the chapter for your problem family. Before running a model, read its assumptions: two models using the same words may represent different systems. Do not confuse deterministic inventory with random demand, an acyclic route with a general network, or a local optimum with a global one.

2.2 The modeling cycle

A rigorous study records:

1. the real system, time horizon, and decision to be supported;
2. controllable variables, parameters, and units;
3. assumptions, uncertainty, and the scope of the data;
4. objective, constraints, and initial conditions;
5. solution method, backend, and execution options;
6. feasibility checks and interpretation;
7. sensitivity, risks, and limits of application.

Write down the unit of every parameter. If demand is expressed per year and lead time in days, convert them to a common basis before calculating. A number without a unit can produce a mathematically consistent solution that is operationally incorrect.

2.3 Validity and exactness

Validation checks structure, dimensions, fields, and family-specific rules. It guarantees neither feasibility nor appropriate assumptions. A solver may prove optimality for a model, return a

heuristic solution, or stop at a limit. Consult the status, algorithm, exactness, and execution notes.

In the metadata, `exact` characterizes the procedure for the supported problem; it does not imply symbolic arithmetic or infinite precision. `fixtureScale` identifies an educational method intended for small cases. Numerical solution uses tolerances; compare results within a margin consistent with their scale rather than requiring decimal strings to match exactly. Repeatability does not prove validity either: a formulation error can be perfectly deterministic.

2.4 Computational traceability

To reproduce an analysis, preserve the input JSON, QSB version, command, backend, options, and result. Also record any transformations from a spreadsheet. For external data, include the source, license, date, and unit of analysis. A solution document does not always contain the complete model; save it separately.

The commands in this manual run from the repository root. `swift run qsb` compiles if necessary; `.build/debug/qsb` uses the built executable. Small examples let you check results by hand before studying larger cases. Changing an input requires validation and solution again, rather than merely editing a presentation table.

Remark 2.1. The computational result expresses the consequences of the encoded data and assumptions. The analyst must justify those assumptions, compare the solution with the real system, and communicate its limitations to the decision maker.

2.5 Comparison and sensitivity

Distinguish verification (the program computes what was specified) from model validation (what was specified adequately represents the system). Check balances and constraints using the returned values. Then vary demand, costs, or probabilities while keeping their distributions and units consistent. A change in recommendation after a small perturbation may reveal a fragile decision; a tie may require secondary criteria outside the model. Do not extrapolate a conclusion beyond the range analyzed.

Part II

Optimization and decision making

Chapter 3

Linear programming

3.1 What a linear model represents

Linear programming (LP) allocates scarce resources among activities when both the objective and the constraints are linear [Dantzig, 1963, Bertsimas and Tsitsiklis, 1997]. A variable may represent units produced, hours assigned, or quantities transported. Its meaning and unit must be established before writing coefficients.

Definition 3.1. A continuous linear model has the form

$$\operatorname{opt}_{x\in\mathbb{R}^n} c^\top x, \qquad a_i^\top x \ \{\leq, =, \geq\}\ b_i \quad (i = 1, \ldots, m), \qquad \ell_j \leq x_j \leq u_j.$$

The operator denotes minimization or maximization. A bound may be absent; a free variable admits both positive and negative values.

Linearity assumes proportionality and additivity: doubling an activity doubles its resource use and contribution, and contributions add without interaction effects. A continuous formulation allows divisibility. The data are also deterministic over the modeled horizon. Discounts, start-ups, indivisible batches, or variable yields require a different formulation. Validating a file checks its structure and mathematical rules; it does not establish that these assumptions accurately represent the organization.

3.2 Example: product mix

A plant manufactures two divisible products, X_1 and X_2. Their unit contributions are 50 and 60 monetary units. Each unit uses two resources as shown in the following table; capacities cover the same period as production. The model is

Table 3.1: Product mix data

Resource	Per unit of X_1	Per unit of X_2	Capacity
Resource 1	2	3	180
Resource 2	3	2	150

$$\begin{aligned} \max \quad & 50X_1 + 60X_2 \\ \text{subject to} \quad & 2X_1 + 3X_2 \leq 180, \\ & 3X_1 + 2X_2 \leq 150, \\ & X_1, X_2 \geq 0. \end{aligned}$$

Capacities are hard constraints: exceeding them is not allowed. Minimum demand requirements and fixed costs are not included; attributing them to this result would change the problem.

3.2.1 Reproducible input and execution

The complete model is in `manual/examples/lp_sample.json`.
The keys `sense`, `variableNames`, `objectiveCoefficients`, and `constraints` define the objective and rows. Each row includes a name, coefficients in variable order, a relation, and `rhs`. Bounds are `lowerBounds` and `upperBounds`; `null` in an upper bound means that there is no explicit maximum. The example uses `continuous` types. Run from the repository root:

```
swift run qsb solve-json manual/examples/lp_sample.json --backend native
```

This specific command returns `objectiveValue` and the `variableValues` dictionary. It does not include a sensitivity report.

On macOS, open the normalized JSON in the `Linear / Integer Programming` family. The `Definition` view shows `Model`, `Variables and objective`, and the constraints. To enter it manually, select `Maximize`, create two variables with coefficients 50 and 60, type `Continuous`, a lower bound of zero, and no upper bound; then enter the two rows with relation $\leq$. Use `Validate` and correct the diagnostics before pressing `Run LP`. The editor shortcuts are Command–Shift–V to validate and Command–R to solve. `Copy Table` and `Paste Table` transfer tables to and from a spreadsheet. Preserve the column order and headers; review the model after pasting. In narrow windows, the editor may require horizontal scrolling.

3.2.2 Result and interpretation

Table 3.2: Linear optimum verified with QSB

Quantity	Value
X_1	18
X_2	48
Total contribution	3780

The solution view shows the objective, variables, and constraint details. Check feasibility numerically:

$$2(18) + 3(48) = 180, \qquad 3(18) + 2(48) = 150.$$

Both resources are fully used; their slacks $b_i - a_i^\top x$ are zero. An active constraint holds as an equality at the optimum; a positive slack indicates unused capacity. This interpretation depends on the relation: for $a_i^\top x \geq b_i$, the surplus is $a_i^\top x - b_i$.

The feasible region is a polygon with vertices $(0, 0)$, $(50, 0)$, $(18, 48)$, and $(0, 60)$. Their objective values are 0, 2500, 3780, and 3600. This independent geometric check confirms the result for the two-variable problem. In higher dimensions, enumerating vertices by hand is impractical.

3.3 Duality and sensitivity: what can be inferred

For this maximization problem with constraints $Ax \leq b$, $x \geq 0$, the dual is

$$\min\ 180y_1 + 150y_2, \qquad 2y_1 + 3y_2 \geq 50, \quad 3y_1 + 2y_2 \geq 60, \quad y_1, y_2 \geq 0.$$

Since both production variables are positive at the optimum, both dual constraints hold as equalities. They yield $y_1 = 16$, $y_2 = 6$, and $180(16) + 150(6) = 3780$. The dual values certify an upper bound equal to the achieved objective.

As long as this basis remains optimal, one additional unit of resource 1 is worth 16 monetary units, and one of resource 2 is worth 6. These are local marginal values, not prices valid for every expansion. For example, changing only the first capacity to $180 + \delta$ moves the intersection to

$$X_1 = 18 - \tfrac{2}{5}\delta, \qquad X_2 = 48 + \tfrac{3}{5}\delta.$$

Both quantities remain nonnegative for $-80 \leq \delta \leq 45$, the interval over which the objective is $3780 + 16\delta$. At the endpoints the basis structure changes; outside the interval the problem must be solved again. This derivation is an academic calculation for the example: QSB currently displays neither shadow prices nor sensitivity ranges in the specific LP output or its native view.

3.4 Method, backends, and limitations

The `native` backend uses two-phase simplex: it first seeks a feasible solution and then improves the objective. It is deterministic and educational, using floating-point arithmetic. A mathematical optimum does not imply symbolic precision. Widely differing coefficient scales can impair calculations; use consistent units and check residuals.

`validate` runs the checks without computing an optimum. `external` requests HiGHS, installed independently and discovered through `QSB_HIGHS_PATH` or `PATH`; if it is unavailable, the request fails explicitly. There is no silent fallback to the native backend. The educational engine does not provide the preprocessing tools or scale guarantees of an industrial engine. To exchange the model with another optimizer, you can generate MPS:

```
swift run qsb export-mps manual/examples/lp_sample.json > /tmp/qsb-lp.mps
```

MPS preserves names through a deterministic adaptation and expresses bounds and domains; JSON remains QSB's normalized representation.

Distinguish three situations: an invalid model contains inconsistent data; an infeasible model has no decision satisfying all constraints; an unbounded model admits unlimited improvements in the objective. Adding an arbitrary bound to make an error disappear changes the model's meaning. Review the units, signs, and limits that the case actually imposes.

3.5 Exercises

1. Increase the capacity of resource 1 to 190. Before running, predict $X_1 = 14$, $X_2 = 54$, and objective 3940 using the sensitivity analysis above; confirm the constraints with QSB.

2. Reduce the capacity of resource 2 to 120 and solve. Explain why a marginal valuation should not be extrapolated indefinitely.

3. Add $X_1 \geq 60$. Identify the contradiction with resource 2 and compare a structurally valid model with an infeasible problem.

4. Change the variables to integers and study the next chapter. Why does the optimum remain unchanged in the original example?

Chapter 4

Integer and mixed-integer programming

4.1 Indivisible decisions

Integer programming requires some decisions to take discrete values: selected projects, shifts, batches, or open facilities. A mixed-integer linear model retains a linear objective and linear constraints but combines continuous and integer variables [Nemhauser and Wolsey, 1988] and [Wolsey, 1998]. Integrality is part of the model, not a presentation operation on the result. Rounding a continuous optimum can violate constraints or lose optimality.

An integer variable represents indivisible quantities; a binary variable represents a yes/no decision. In QSB the normalized types are `continuous`, `integer`, and `binary`. All share the coefficients, relations, and bounds of the linear model. In a selection model, a binary y_j can activate a continuous quantity x_j through $0 \leq x_j \leq M_j y_j$. The constant M_j must be a justifiable bound: an excessively large number weakens the relaxation and can impair numerical stability.

4.2 Example: minimum coverage with integer activities

Two activities provide coverage for two requirements. Each unit of the first costs 2.5 monetary units, and each unit of the second costs 2. Their contributions are $(6, 3)$ and $(3, 5)$; at least 200 units of the first requirement and 180 of the second are needed. Activities can only be purchased in whole units:

$$\begin{aligned} \min \quad & 2.5X_1 + 2X_2 \\ \text{subject to} \quad & 6X_1 + 3X_2 \geq 200, \\ & 3X_1 + 5X_2 \geq 180, \\ & X_1, X_2 \in \mathbb{Z}_+. \end{aligned}$$

Cost is additive and contributions remain constant: discounts and changes in productivity have not been modeled. The complete case is in `manual/examples/ilp_sample.json`; both variables have `variableTypes` set to `integer` and a lower bound of zero.

4.2.1 CLI and macOS execution

From the repository root:

```
swift run qsb solve-json-ilp manual/examples/ilp_sample.json --backend native
```

Use the specific ILP command to respect the integer domains; solving only the relaxation does not answer the indivisible decision problem.

On macOS, use the same `Linear / Integer Programming` editor as in Chapter 3. Open the JSON or enter the two variables with `Minimize`, coefficients 2.5 and 2, type `Integer`, a lower bound of zero, and no explicit upper bound. Enter the two constraints with relation $\geq$, validate, and press `Run ILP`. The button label changes according to the variable types. The solution displays quantities and constraint evaluations; retain the exported JSON to review the original domains.

4.2.2 Optimum and verification

QSB returns $X_1 = 22$, $X_2 = 23$, and cost 101. Feasibility is verified by

$$6(22) + 3(23) = 201 \geq 200, \qquad 3(22) + 5(23) = 181 \geq 180.$$

Coverage surpluses are one unit for each requirement. The cost is $2.5(22) + 2(23) = 101$, and both quantities are integers.

To understand the difference from the relaxation, temporarily allow continuous variables. The intersection of the two boundaries is

$$X_1 = \frac{460}{21} \approx 21.904762, \qquad X_2 = \frac{160}{7} \approx 22.857143,$$

with cost $2110/21 \approx 100.476190$. In minimization, relaxing integrality can only reduce the optimal cost or leave it unchanged; it provides a lower bound for the integer problem. The absolute difference is $11/21 \approx 0.523810$. Rounding up happens to produce the optimal integer solution here, but this is a coincidence of the case, not a general rule.

There is also a short verification of the integer cost. For any feasible solution, the cost is at least $2110/21$ and lies on the grid of multiples of 0.5. Ruling out cost 100.5 is therefore sufficient to prove that 101 is the minimum. If the cost were 100.5, then $5X_1 + 4X_2 = 201$. Substitution into the two coverage requirements simultaneously yields $X_1 \geq 197/9$ and $X_1 \leq 285/13$; the narrow interval $[21.888889, 21.923077]$ contains no integer. Thus, cost 100.5 is unattainable, and QSB's solution certifies the next possible value, 101.

4.3 Branch and bound

The native algorithm solves an LP relaxation at each node. If the solution is integral, it becomes a candidate. If a variable takes a fractional value v, two problems are created, $x_j \leq \lfloor v \rfloor$ and $x_j \geq \lceil v \rceil$, covering its integer possibilities. An infeasible branch is discarded; a branch whose bound cannot improve the best known solution is also discarded. Exploring all relevant branches establishes optimality within numerical tolerances.

QSB uses a deterministic search and a default limit of 10000 nodes. If this is exceeded, it reports a model unsupported by that search; this must not be interpreted as an optimal solution or proof of infeasibility. The educational backend lacks the cutting-plane and preprocessing techniques of an industrial MILP engine. Running time can grow rapidly even for a small file; providing valid physical bounds can improve the formulation but must never exclude feasible decisions.

`validate` checks the model without solving it. `external` uses the HiGHS executable discovered on the computer for LP/MIP and requires an independent installation. The selected backend does not change the meaning of the variables; if unavailable, the request fails. The `export-mps` command accepts this JSON and expresses its integer domains, bounds, and linear rows. This MILP capability does not imply that HiGHS supports mixed-integer quadratic programming; these are different classes.

4.4 Interpretation and common mistakes

Compare costs only between feasible solutions of the same model. The relaxation bound measures the idealized cost when fractions are allowed, but does not represent an implementable alternative. In binary models, check that each value represents the intended action; in mixed models, distinguish continuous quantities from activation decisions. A fractional solution obtained by changing the types to continuous answers a different problem.

Do not multiply a binary variable by a continuous variable within an LP row: that is a nonlinear expression. Activation rules require auxiliary linear constraints. Likewise, do not convert a continuous variable to integer simply to make the screen display round numbers: the domain reflects the decision unit.

4.5 Exercises

1. Duplicate the example JSON and change both types to `continuous`. Solve with `solve-json` and contrast the lower bound calculated above with the integer result.

2. Impose $X_1 \leq 21$. Calculate the minimum X_2 satisfying both coverage requirements, and check the new cost with QSB.

3. Formulate a selection problem with three projects, costs 4, 6, and 7, benefits 5, 8, and 10, budget 10, and binary variables. Determine the optimal combination by enumeration, then verify it.

4. Explain why rounding down both quantities in the example relaxation violates the requirements. What would change if the constraints were maximum capacities?

Chapter 5

Goal, quadratic, and nonlinear programming

This chapter brings together three distinct extensions of linear programming. Goals order multiple objectives; quadratic programming adds curvature to the objective while retaining linear constraints; nonlinear programming allows general expressions. Choosing a family changes both the formulation and the guarantee the solver can offer. Comparing the number displayed as the objective is insufficient: priorities, feasibility, and exactness must also be reviewed.

5.1 Goal programming

5.1.1 Priorities and deviations

An organization may first seek to meet a goal and then optimize another quantity without sacrificing what has already been achieved. In lexicographic programming, $g_1(x)$ is optimized, its optimal value is fixed, and $g_2(x)$ is optimized within that set; the process continues in the declared order. Objectives are not automatically added with large weights. The order expresses a policy decision: an improvement in a later priority never compensates for deterioration in an earlier one.

For a goal $a^\top x = t$, the deviation variables satisfy

$$a^\top x + n - p = t, \qquad n, p \geq 0.$$

n measures shortfall and p surplus. Minimizing $n + p$ penalizes both directions; minimizing only n tolerates surplus. Coefficients and units must express the intended penalty. QSB does not infer these equations: the user declares variables, objectives, and constraints.

5.1.2 Example: protecting a production commitment

There is capacity for ten units shared between products A and B. The first aim is to bring A close to eight units, followed by maximizing B:

$$\begin{aligned} &\text{Priority 1: } \min(n_1 + p_1), \\ &\text{Priority 2: } \max B, \\ &A + B \leq 10, \qquad A + n_1 - p_1 = 8, \\ &A, B, n_1, p_1 \geq 0. \end{aligned}$$

The priority does not itself require exact fulfillment if the goal is impossible; it minimizes deviation subject to the hard constraints. In this example, zero deviation is attainable. Once it is fixed, $A = 8$, so the subsequent maximum is $B = 2$.

The file `manual/examples/opt_goal.json` contains all variables, the two rows, and the ordered `goals` list; each priority has a name, `sense`, and `coefficients`. Bounds and types are declared in arrays aligned with `variableNames`. Run:

```
swift run qsb validate-goal-json manual/examples/opt_goal.json
swift run qsb solve-goal-json manual/examples/opt_goal.json --backend native
```

The output document retains the model, `backend` metadata, and solution.

`goalOutcomes` displays priority, sense, and value. Capacity is active and the goal is met.

Table 5.1: Lexicographic result for the production case

Quantity	Value
Priority 1: total deviation	0
Priority 2: production of B	2
A	8
n_1, p_1	0, 0

Reversing the order makes the first objective yield $B = 10$; with that value fixed, $A = 0$, $n_1 = 8$, and deviation is eight. This contrast invites discussion of who determines precedence and which commitment it protects.

5.1.3 macOS workflow and solver character

Open the normalized file in `Goal Programming`. In `Definition`, review variables and bounds, the priority list, and linear constraints. The move-up and move-down controls change a priority's position. Enter the deviation equation as an equality and verify the negative sign of p_1. Press `Validate` and `Run`; the native priority view shows the ordered objectives, final quantities, and backend information. JSON provides the complete exportable record. Tables support copying and pasting; coefficient columns must match the current variable order. Large tables use row selection and coefficient groups.

The native backend solves a sequence of LP or integer problems, adding an equality to preserve each previous optimum. The continuous case is labeled `exact`; this term characterizes the method, subject to floating-point tolerances. Integer goals use an educational search for small problems, identified as `fixtureScale`, and depend on branch-and-bound limitations. The special relaxation of integer deviation pairs applies only when integral equations justify integral residuals; it does not remove integrality from the decisions.

`validate` does not solve. `external` executes each priority with HiGHS if installed; QSB preserves the lexicographic order and fixing. A missing executable causes an explicit error. An infeasible later priority, an unbounded objective, or exhausted limits require reviewing the diagnostic rather than interpreting partial results as the final plan.

5.2 Quadratic programming

5.2.1 Objective convention and assumptions

QSB represents the objective as

$$f(x) = x^\top Q x + c^\top x,$$

with linear constraints, bounds, and variable types. JSON uses `quadraticMatrix` for Q and `linearCoefficients` for c. This convention has no factor of $1/2$. For a symmetric matrix, the

coefficient of the product $x_i x_j$, $i \neq j$, is $2Q_{ij}$: a term $4XY$ is represented by $Q_{12} = Q_{21} = 2$. Do not place 4 in both cells, since that would double the effect.

A symmetric positive-definite matrix produces a strictly convex objective; its minimum over a convex region, if it exists, is unique. A negative-definite matrix produces the equivalent strictly concave maximization case. Stationarity and feasibility conditions then identify the global optimum. An indefinite matrix changes the problem and is not covered by that guarantee.

5.2.2 Example: adjustment with limited capacity

Two quantities should be brought close to 2 and 3, respectively, with total capacity four. Removing the constant 13 from the squared distance gives

$$\begin{aligned} \min \quad & X^2 + Y^2 - 4X - 6Y \\ \text{subject to} \quad & X + Y \leq 4, \qquad X, Y \geq 0. \end{aligned}$$

The matrix is the identity and the linear vector is $(-4, -6)$. Removing the constant preserves optimal quantities but changes the reported objective value: the original squared distance is $f(X, Y) + 13$.

The model is in `manual/examples/opt_qp.json`. Run:

```
swift run qsb validate-qp-json manual/examples/opt_qp.json
swift run qsb solve-qp-json manual/examples/opt_qp.json --backend native
```

The optimum is $X = 1.5$, $Y = 2.5$, $f = -12.5$, with `Capacity` in `activeConstraints`. The original squared distance is 0.5. A negative objective is perfectly valid: it results from the omitted constant and does not represent a negative distance.

Without the capacity constraint the solution would be $(2, 3)$, which is infeasible. With multiplier $\lambda \geq 0$, the stationarity equations are

$$2X - 4 + \lambda = 0, \qquad 2Y - 6 + \lambda = 0.$$

The active capacity requires $X + Y = 4$, giving $\lambda = 1$, $X = 1.5$, and $Y = 2.5$. The lower bounds are inactive. Strict convexity and the conditions above certify the global minimum of the continuous model. QSB displays active constraints, but this multiplier derivation is an independent academic check.

5.2.3 macOS workflow and limitations

Open the JSON in `Quadratic Programming`. `Definition` separates variables and bounds, the quadratic matrix, and linear constraints. Enter the linear coefficients in the variable rows and the identity in the matrix; confirm the names on both axes. Add `Capacity` with coefficients $(1, 1)$, relation $\leq$, and right-hand side 4. Validate and press `Run`. The native solution displays the objective, variables, bounds, active constraints, and exactness; JSON allows inspection of the document.

`Copy Table` and `Paste Table` transfer the complete matrix. Reordering known variables preserves both axes by name; pasted headers must match the current order. For more than twelve variables, the editor displays one row and groups of columns, although table exchange remains complete. This editor capability does not imply support for large problems in the native solver.

The continuous native solver enumerates active sets and solves KKT systems for strictly convex minimization or strictly concave maximization. Its label is `exact`, with numerical calculations. The search rejects more than twenty candidate inequalities, counting bounds

that it converts into inequalities; the cost of set enumeration grows rapidly. The integer native solver supports only diagonal concave maximization with nonnegative variables, up to four variables, and enumerates points within derived finite bounds. It is identified as `fixtureScale`; it is not a general MIQP solver.

The external HiGHS backend supports translation of the supported strictly convex/concave continuous cases. QSB adapts the convention using a Hessian $H = 2Q$, because the external solver uses $\frac{1}{2}x^\top Hx$. The adaptation also preserves sense, bounds, and constraints, and verifies the objective and feasibility when reading the solution. HiGHS does not solve MIQP in this integration; quadratic integer variables require the native scope described above or a future engine. The `validate` mode does not compute an optimum.

5.3 Nonlinear programming

5.3.1 Expressions and domain

Nonlinear programming (NLP) allows an objective $f(x)$ and constraints $g_i(x)$ $\{\leq, =, \geq\}$ b_i. QSB accepts textual expressions, ordered variables, and nullable bounds. The evaluator recognizes addition, subtraction, multiplication, division, powers, parentheses, implicit multiplication, and `exp`, `log`, `sqrt`, `sin`, `cos`. Use explicit multiplication in teaching materials to aid reading. Names are recognized without case sensitivity; consistent spelling is advisable.

Valid syntax does not guarantee a valid domain at every point: `log(X)` requires $X > 0$, `sqrt(X)` requires $X \geq 0$, and division requires a nonzero denominator. Bounds must correspond to the case and protect the domain where possible. Sine and cosine angles are expressed in radians. JSON stores `objectiveExpression` and, for each constraint, `expression`, `relation`, and `rhs`.

5.3.2 Example: constrained distance

The file `manual/examples/opt_nlp.json` specifies

$$\min\ (X-3)^2 + (Y-2)^2, \qquad X + Y \leq 4, \qquad 0 \leq X, Y \leq 4.$$

The objective is the squared distance to point $(3, 2)$, which violates capacity. Its projection onto $X + Y = 4$ is $(2.5, 1.5)$, and the mathematical value is 0.5. Although this case could be formulated as QP, it serves to compare expression input and an approximate method with a known analytical solution.

```
swift run qsb validate-nlp-json manual/examples/opt_nlp.json
swift run qsb solve-nlp-json manual/examples/opt_nlp.json --backend native
```

The verified execution returns approximately $X = 2.500000002$, $Y = 1.499999999$, objective 0.4999999995, and maximum violation 5.0×10^{-10}. The evaluation of `Capacity` is approximately 4.0000000005. The `iterations` field records 361 cumulative iterations of the selected search; it does not count all evaluations of searches from the other initial points. This value describes this execution, not a universal limit or a measure comparable with simplex iterations.

The small violation does not authorize physically exceeding capacity. It is a numerical search residual; for real implementation, check the values against the tolerance and units of the case. The output contains `constraintEvaluations`, `maximumViolation`, `iterations`, the objective, and variable values, as well as the model and metadata. Reporting the objective without the residual would hide relevant information.

5.3.3 macOS use and approximate guarantee

Open the JSON in `Nonlinear Programming`. The definition allows editing the sense, objective expression, bounded variables, and constraints with an expression, relation, and right-hand side. Enter `(X-3)^2+(Y-2)^2`, bounds 0 and 4, and `X+Y` with ≤ 4. Use `Validate` before `Run`. An invalid expression is retained for correction. Variables and constraints support spreadsheet exchange; each valid paste replaces a complete table. The solution displays the candidate, bounds, evaluations, and maximum violation, together with its approximate character and exportable JSON.

The native backend uses automatic differentiation, searches from multiple points, and progressive constraint penalties. It is deterministic, is limited to four variables, and is identified as `approximate`. It also includes a low-dimensional search on equality manifolds for small cases. These mechanisms do not prove global optimality for nonconvex functions. A feasible candidate may be local, and a failed search does not in itself constitute a mathematical proof of infeasibility.

Strict inequalities from imports are normalized to non-strict boundaries and generate a structured warning. The `normalizedStrictInequalities` field records that transformation; it must be reviewed if the boundary has a physical meaning excluded by the case. `validate` diagnoses without solving. `external` is explicitly unavailable for NLP; select the native backend for supported educational problems.

5.4 How to compare and review results

Retain each input together with its output and the backend used. For goals, review the order and each fixed value. For QP, check the matrix convention and supported curvature. For NLP, review domains, bounds, evaluations, and maximum violation before describing a solution as implementable. File validation and search convergence provide different kinds of evidence.

All three families have a native definition view and a JSON view; unfinished fields remain editable when validation fails. In native macOS runs, solving takes place in the background and duplicate runs are prevented while one is in progress. Changing the model can make that calculation's result stale. There is currently no cancellation control for these workers. Accessible labels on controls and results do not replace a complete VoiceOver and keyboard navigation audit, which remains pending for these editors.

5.5 Exercises

1. Reverse the priorities in the goal example. Verify $B = 10$, $A = 0$, and total deviation 8. Explain which commitment has changed.

2. Reduce the capacity in the goal example to 6. Calculate the minimum deviation of the first priority and justify why the second cannot recover production of B without worsening the first.

3. In the QP, increase capacity to 5. Predict $(X, Y) = (2, 3)$, objective -13, and zero original distance. Check against QSB.

4. Introduce a term $4XY$ in a test objective. Write the correct symmetric matrix and examine its curvature before choosing a solver.

5. Reformulate the example NLP objective in QP form, retaining constraints and bounds. Compare quantities and residuals; explain the difference between its objective with a constant and the quadratic objective without it.

6. Propose a one-variable function with two local minima. Explain what additional evidence you would need to attribute global optimality to a candidate obtained by the native NLP solver.

Chapter 6

Network models

6.1 From a situation to a graph

A network represents places, states, or people as nodes; arcs represent possible connections. The meaning of the number on an arc depends on the model: cost or distance in a route, capacity in a maximum-flow problem. Confusing these quantities changes the problem [Ahuja et al., 1993]. The examples in this chapter are original, and their files are in `manual/examples/`. They all use the JSON envelope with `kind` and `model`; WinQSB files are not required. Run the commands from the repository root.

6.2 Shortest path and minimum spanning tree

SPP seeks a directed path from the source to the destination with minimum cost. Dijkstra requires nonnegative costs. In QSB, the first node is the source and the last is the destination. An absent arc is not equivalent to a zero-cost arc. The file `plan_spp.json` has nodes A, B, C, D and the following arcs:

Arc	A–B	A–C	B–C	B–D	C–D
Cost	2	5	1	6	2

```
swift run qsb validate-network-json manual/examples/plan_spp.json
swift run qsb solve-network-json manual/examples/plan_spp.json --backend native
swift run qsb solve-network-json manual/examples/plan_mst.json --backend native
```

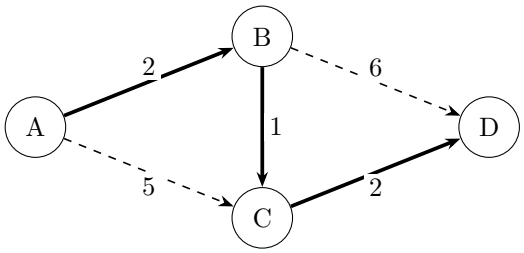


Figure 6.1: Shortest-path network: labels are costs. Thick arrows trace the optimal route; dashed arrows show alternatives.

SPP returns A → B → C → D with cost $2 + 1 + 2 = 5$. MST connects all nodes without cycles at the lowest total cost; it does not seek a path between endpoints. The example uses the same connections as undirected edges, and Kruskal chooses B–C, A–B, C–D, with cost 5. QSB normalizes asymmetric MST connections to the lower cost for each pair; a disconnected network has no tree spanning all nodes.

6.3 Maximum flow

With capacities u_{ij}, flow must satisfy $0 \le x_{ij} \le u_{ij}$ and conservation at intermediate nodes. The objective maximizes net flow leaving the source. It does not minimize cost. In `plan_maxflow.json`, S–A has capacity 3, S–B 2, A–B 1, A–T 2, and B–T 3. Although the JSON field is named `cost`, it represents capacity in MFP.

```
swift run qsb solve-network-json manual/examples/plan_maxflow.json --backend native
```

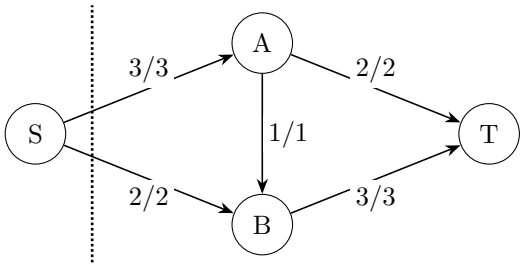


Figure 6.2: Maximum flow: each label gives flow/capacity. The dotted line separates the source from the remaining nodes and marks a saturated cut.

Edmonds–Karp obtains flow 5: 3 and 2 units leave S, and 2 and 3 reach T. The cut separating S from the other nodes has capacity 5 and proves optimality. Again, the first and last nodes determine the endpoints.

6.4 Traveling salesperson

TSP seeks a directed cycle that visits every node exactly once and returns to the starting node. Connecting the nodes is insufficient: there must be a single tour. The example `plan_tsp.json` contains three nodes, with $c_{AB} = 2$, $c_{BC} = 3$, $c_{CA} = 4$ and reverse costs 4, 5, and 6.

```
swift run qsb solve-network-json manual/examples/plan_tsp.json --backend native
```

The tour A $\to$ B $\to$ C $\to$ A costs 9; the reverse tour costs 15. Held–Karp is exact for the accepted model but exponential; the solver rejects more than 20 nodes. A model with no feasible Hamiltonian cycle is not repaired by adding connections. The `fixtureScale` label expresses this scale limitation, not a guarantee of fast execution near the maximum.

6.5 Assignment and transportation

Assignment minimizes $\sum_{ij} c_{ij} x_{ij}$, assigning each worker to one task and using each task at most once. The Hungarian method accepts rectangular matrices with at least as many tasks as workers. In `plan_assignment.json`, the rows Ana and Luis have costs $(2, 7, 4)$ and $(6, 1, 3)$ for X, Y, and Z.

```
swift run qsb solve-network-json manual/examples/plan_assignment.json --backend
    native
swift run qsb solve-network-json manual/examples/plan_transport.json --backend
    native
```

Ana–X and Luis–Y cost 3; Z remains unassigned. The model does not include teams, multiple tasks per person, or additional constraints.

Transportation allows continuous quantities and requires balance:

$$\min \sum_{ij} c_{ij}x_{ij}, \qquad \sum_{j} x_{ij} = s_i, \quad \sum_{i} x_{ij} = d_j, \quad x_{ij} \geq 0.$$

The example has supply $(4, 6)$, demand $(5, 5)$, and cost matrix $\left(\begin{smallmatrix} 1 & 4 \\ 3 & 2 \end{smallmatrix}\right)$. QSB ships 4 from O1 to D1, 1 from O2 to D1, and 5 from O2 to D2; the cost is $4+3+10 = 17$. Check row and column sums separately. An imbalance is a validation error in TP. This family does not automatically create a dummy source or destination.

6.6 Minimum-cost flow

CNF combines transshipment and costs: at each node, $\sum_j x_{ij} - \sum_j x_{ji} = s_i - d_i$, and it minimizes $\sum_{ij} c_{ij}x_{ij}$ with nonnegative continuous quantities. The current model has no arc capacities or lower bounds. The validator requires finite nonnegative costs in CNF and the graph variants; assignment permits finite negative costs. Do not model a subsidy as a negative cost on a CNF arc, because it will be rejected. `plan_netflow.json` supplies 4 at O, requires 4 at D, and allows O–M at cost 1, M–D at cost 2, and O–D at cost 5.

```
swift run qsb solve-network-json manual/examples/plan_netflow.json --backend native
```

The 4 units pass through M and cost 12; direct shipment would cost 20. CNF preserves original supply and demand and, when they differ, emits a warning and adds a zero-cost dummy adjustment during solving. Always review `balanceAdjustments`: a solution with an adjustment does not prove that original supply can meet actual demand.

6.7 macOS use and backend selection

Open the example by importing normalized JSON. In Definition, SPP, MST, MFP, and TSP have a node and arc editor; AP and TP have cost matrices, with supply and demand in TP; CNF has nodes, arcs, and balances. Changes are validated before Run. The endpoint order for SPP/MFP maps to the JSON order; visual positions are not exported. The solution Diagram highlights the path, tree, tour, or connections used. `All connections` also shows the remaining connections. In bipartite networks, the groups represent workers/tasks or sources/destinations. Use the detail panel to read exact values and JSON to audit the result; the thickness of a connection does not replace its numerical quantity.

Native offers deterministic algorithms for all seven variants. Validate checks the model without solving it. External is available for TP and CNF through HiGHS; there is no external backend covering all network variants. Solutions preserve the model, algorithm, assumptions, and exactness. The practical size of native LPs depends on the formulation; for large transportation networks, consider HiGHS and check its availability.

6.8 Exercises

1. Change A–C from 5 to 1 in SPP. Verify the new path and explain why the minimum spanning tree may respond differently.

2. Reduce S–A to 1 in MFP. Calculate a cut before solving.

3. Change D2 demand to 6 in TP. Observe validation; then deliberately represent one additional unit of supply.

4. In CNF, increase demand to 5. Interpret the dummy adjustment and explain which business data are missing before the plan can be accepted.

Chapter 7

Decisions, games, and dynamic programming

7.1 Choosing under uncertainty

A decision combines controllable alternatives with states of the environment that the analyst does not control. A payoff table organizes the payoff a_{ij} from alternative i in state j. When prior probabilities p_j are known, the expected-value criterion selects

$$i^* \in \arg\max_i \sum_j p_j a_{ij}.$$

This criterion assumes comparable payoffs and risk neutrality. A solution maximizing average profit may involve substantial losses in a particular state. QSB does not automatically transform money into utility or incorporate the decision maker's risk aversion.

7.1.1 Payoff tables and information

The file `dec_payoff.json` considers a launch with demand Alta (High, probability 0.4) or Baja (Low, 0.6). The input's Spanish identifiers are retained here so that they match the files and output: Lanzar means Launch, Esperar means Wait, Positivo means Positive, and Negativo means Negative. The payoffs are:

Decision	Alta	Baja
Lanzar	100	−40
Esperar	20	10

Without a market study, Lanzar yields $0.4(100) + 0.6(-40) = 16$, compared with 14 for Esperar. With perfect information, Lanzar would be chosen in Alta and Esperar in Baja: $0.4(100) + 0.6(10) = 46$. The expected value of perfect information (EVPI) is $46 - 16 = 30$, an upper bound on the price of information under this model.

The study produces a Positivo or Negativo indicator. The `indicatorLikelihoods` matrix uses indicator rows and state columns: $P(+ \mid Alta) = 0.8$ and $P(+ \mid Baja) = 0.2$. Each column represents a distribution over possible indicators. These are not posterior probabilities. The policy with the study is Lanzar if the result is Positivo and Esperar if it is Negativo. Its expected value is 33.6; hence EVSI (expected value of sample information) is $33.6 - 16 = 17.6$. This amount does not deduct the cost of the study. If the study cost 20, purchasing it would not be worthwhile under the expected-value criterion.

```
swift run qsb validate-decision-json manual/examples/dec_payoff.json
swift run qsb solve-decision-json manual/examples/dec_payoff.json --backend native
```

The output contains `priorExpectedValues`, `indicatorAnalyses`, `bestPriorDecision`, EVSI, and EVPI. Keys retain their English names. The solver requires a probability greater than 10^{-12} for every indicator; otherwise, solution fails rather than inventing a posterior.

7.1.2 Bayesian updating

Bayes' rule converts prior probability and likelihood into posterior probability:

$$P(s_j \mid e_k) = \frac{P(e_k \mid s_j)p_j}{\sum_\ell P(e_k \mid s_\ell)p_\ell}.$$

The file `dec_bayes.json` separates this operation from any payoff table. It uses the same states and study. QSB obtains $P(+) = 0.44$, $P(-) = 0.56$, and the following posteriors:

Evidence	Alta	Baja
Positivo	0.727273	0.272727
Negativo	0.142857	0.857143

```
swift run qsb validate-decision-json manual/examples/dec_bayes.json
swift run qsb solve-decision-json manual/examples/dec_bayes.json
```

`likelihoods` again uses outcome rows and state columns. Changing the order of names requires changing the corresponding rows or columns as well. Probabilities must be finite, nonnegative, and consistent with the distributions represented. Bayes updates beliefs; by itself it does not say which action to take.

7.1.3 Trees and backward induction

A tree makes the temporal order of decisions and observations explicit. `rootID` identifies the root, each node has a unique `id`, and its `childIDs` identify successors. The types are `decision`, `chance`, and `terminal`. Chance-branch probabilities are stored in the child nodes, rather than in a list on the parent.

Backward induction assigns a terminal its payoff. A decision node adds its local payoff to the maximum value of its children; a chance node adds its local payoff to the expected value of its children. Local payoffs accumulate along the path: do not repeat a cost in the leaves if it was already entered in an ancestor.

In `dec_tree.json`, Invertir (Invest) costs 5 and yields 40 with probability 0.6 or -10 with probability 0.4. Its value is $-5 + 0.6(40) + 0.4(-10) = 15$; Conservar (Retain) yields 12. The root value is 15, and the policy selects Invertir.

```
swift run qsb validate-decision-json manual/examples/dec_tree.json
swift run qsb solve-decision-json manual/examples/dec_tree.json
```

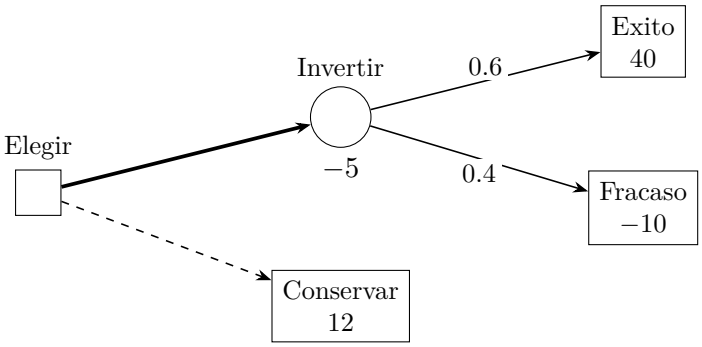


Figure 7.1: Decision tree: the square represents a choice and the circle represents chance. The thick branch is the selected policy. Leaves show payoffs; the local investment cost is added once. Names retain the JSON identifiers.

Review `nodeValues` for each node and `policy` for the decisions. The method maximizes payoffs: to minimize costs, you can use their negatives, explaining the convention. QSB warns about and normalizes chance-child probabilities whose positive sum differs from one, for compatibility with rounded inputs. This repair does not replace a review of the data. Avoid cycles, nonexistent identifiers, and inconsistent probabilities.

7.2 Zero-sum games

In a zero-sum game, the row player's payoff is the column player's loss. The first maximizes and the second minimizes. Mixed strategies are probability distributions over actions; they do not represent fractional quantities of an action in a single play.

For a matrix A, QSB solves equivalent linear programs. The row player seeks $\max v$ subject to $A^T p \geq v\mathbf{1}$, $\sum_i p_i = 1$, and $p_i \geq 0$. The column player solves the dual minimization problem. This assumes simultaneous choices and known payoffs. `dec_game.json` uses

$$A = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Both players assign probability 0.5 to A and B, and the game's value is zero. The numerical output may show 4.44×10^{-16} because of rounding.

```
swift run qsb validate-decision-json manual/examples/dec_game.json
swift run qsb solve-decision-json manual/examples/dec_game.json
```

Read `rowStrategy`, `columnStrategy`, and `value`. The native method uses the shared linear solver. It does not interpret this solution as an equilibrium of a general non-zero-sum game or a sequential game.

7.3 Dynamic programming: state, stage, and action

Dynamic programming divides a problem into stages. The state summarizes the information needed to evaluate future decisions; an action changes that state and produces a local cost or return. The principle of optimality allows an optimal policy to be built from subproblems. A state omitting relevant information breaks that principle.

QSB implements three discrete, deterministic models: bounded knapsack, stage-by-stage acyclic routing, and production with inventory. Inputs use `kind` and `model`. Outputs include a typed result, `assumptions`, and `trace`. Each trace row describes a stage, state, action, next state, and *local* value; this value is not necessarily the subproblem's optimal value function.

7.3.1 Bounded knapsack

Each item i consumes w_i units of capacity and contributes r_i per unit; at most b_i units are available. The formulation is

$$\max \sum_i r_i x_i, \qquad \sum_i w_i x_i \le C, \qquad x_i \in \{0, \dots, b_i\}.$$

A recurrence with remaining capacity c is $F_i(c) = \max_q\{qr_i + F_{i+1}(c - qw_i)\}$, over available quantities that fit. Capacity and consumption are integers; returns may be finite real numbers. Do not convert continuous physical measurements to integers without justifying the discretization unit.

In `dec_knapsack.json`, $C = 7$ and the items (b_i, w_i, r_i) are A $(2, 3, 5)$, B $(3, 2, 3)$, and C $(1, 4, 7)$. QSB selects one A and one C: capacity 7 and return 12. The alternative of one A and two B also uses 7 but returns 11. The selection encodes quantities; the trace also includes the decision to take zero units of B.

```
swift run qsb validate-dp-json manual/examples/dec_knapsack.json
swift run qsb solve-dp-json manual/examples/dec_knapsack.json
```

In macOS, Bounded Knapsack lets you edit capacity, availability, consumption, and returns, and copy or paste the table. Summary provides metrics, bars, and a selection table; the trace helps explain the policy.

7.3.2 Stage-by-stage routing (Stagecoach)

The `nodes` list declares a topological order: the first node is the origin and the last the destination. Every arc must advance in that order and have a finite, nonnegative cost. The method applies

$$V(i) = \min_{(i,j)}\{c_{ij} + V(j)\}, \qquad V(\text{destination}) = 0.$$

`dec_route.json` contains Origen–Norte (4), Origen–Sur (2), Norte–Destino (3), and Sur–Destino (6). The identifiers mean Origin, North, South, and Destination, respectively. The result is Origen–Norte–Destino, with cost 7; choosing the cheapest initial arc would yield 8.

```
swift run qsb validate-dp-json manual/examples/dec_route.json
swift run qsb solve-dp-json manual/examples/dec_route.json
```

A structurally valid graph may have no route to the destination; this is detected during solution. This model does not accept cycles or backward arcs: for other graphs, consult the networks chapter.

7.3.3 Production and inventory

The state at the beginning of a period is inventory I_{t-1}; the action is integer production P_t. Deterministic demand d_t is met with $I_t = I_{t-1} + P_t - d_t \ge 0$. Capacities bound P_t and I_t. QSB assumes zero initial and final inventory, with no backorders. It minimizes

$$\sum_t \left[K_t \mathbf{1}_{P_t>0} + c_t P_t + h_t I_t\right].$$

Holding cost applies to the period's ending inventory, rather than its initial inventory. The recurrence evaluates feasible production quantities and adds the optimal future cost.

`dec_production.json` has demands of 2 and 3, production capacity 5, storage capacity 3, setup cost 10, unit cost 2, and holding cost 1 in both periods. QSB produces 5 initially, carries 3, and produces nothing in the second period. The cost is $10 + 2(5) + 1(3) = 23$, compared

with 30 if production is set up and carried out in each period. The decisions detail initial inventory, production, demand, ending inventory, and local cost.

```
swift run qsb validate-dp-json manual/examples/dec_production.json
swift run qsb solve-dp-json manual/examples/dec_production.json
```

Field validation does not prove the existence of a feasible plan. Insufficient capacities may cause solution failure. This model does not represent multiple products, configurable initial inventory, shortages, or random demand.

7.4 Use, metadata, and shared limits

All four decision variants have native macOS editors. The tree editor allows nodes and links to be edited; removing leaves or adding terminals maintains a structure that is easier to review. Stagecoach and production/inventory can be edited through the structured form or JSON; their results have native route or per-period series views. Use Validation before Run and consult JSON when you need to preserve every field.

The native decision backend declares `exact`: it computes the specified criterion using numerical arithmetic within the model and its educational scale. The three dynamic variants declare `fixtureScale`; their policies are those obtained by the implemented recurrences, with discretization and memory consumption growing with capacity, inventory, and the number of stages. There is no promise of industrial-scale performance. `-backend validate` returns validation rather than a solution; the external backend is unavailable for these families. Save the model alongside the dynamic programming output: its solution document preserves the title and assumptions, but not the complete model.

7.5 Exercises and checks

1. Change $P(Alta)$ from 0.4 to 0.3, adjusting Baja as well. First calculate the best decision without information, then solve again. Why can EVSI change even when the quality of the study remains unchanged?

2. Calculate the maximum price of the example's study and compare it with EVPI. Explain which assumptions would prevent using that price in a real decision.

3. In the tree, increase the cost of Invertir to 9. This branch's value becomes 11, and the root decision changes to Conservar, with value 12.

4. In the knapsack, change capacity to 6. Verify a return of 10 with two units of A, and explain why the return/consumption ratio is insufficient to ensure optimality.

5. In the route, make Sur–Destino cost 4. The route through Sur will cost 6. Then add a backward arc and examine the validation diagnostic.

6. Reduce the production setup cost to 1 in both periods. Producing 2 and 3 separately costs 12, compared with 14 when inventory is carried. Compare each inventory balance and the trace.

Part III

Operations planning

Chapter 8

Inventory models

8.1 The decision being modeled

An inventory policy combines how much to replenish and when to do so. The minimum cost depends on time units, the demand pattern, and shortage costs; entering average demand alone is not sufficient [Harris, 1913, Silver et al., 1998]. QSB distinguishes EOQ/EPQ, quantity discounts, the newsvendor model, period-based lot sizing, and four stochastic review policies. All use the `kind/model` JSON envelope and the native or validation backend. There is no external inventory backend.

8.2 EOQ and gradual production

With constant demand D, fixed ordering cost S, and holding cost H per unit per period, with no shortages, the relevant cost is

$$C(Q) = DS/Q + HQ/2, \qquad Q^* = \sqrt{2DS/H}.$$

Purchasing cost Dc is added to total cost but does not change the optimum when unit price c is constant. Lead time ℓ gives the reorder point $r = D\ell$. Rates, lead time, and holding cost must use the same period. For gradual production at rate $p > D$, average inventory is $Q(1 - D/p)/2$, and

$$Q^*_{EPQ} = \sqrt{\frac{2DS}{H(1 - D/p)}}.$$

The calculation is continuous: purchasing boxes or whole units requires comparing feasible quantities. QSB does not automatically impose packaging constraints.

8.2.1 Example: replenishing notebooks

The file `manual/examples/prob_eoq.json` contains $D = 1200$ units/year, $S = 30$, $H = 2$, $c = 5$, a lead time of 0.01 years, and a current order quantity of 100. From the repository root:

```
.build/debug/qsb validate-inventory-json manual/examples/prob_eoq.json
.build/debug/qsb solve-inventory-json manual/examples/prob_eoq.json
```

The results are $Q^* = 189.736660$, 6.324555 orders/year, a cycle of 0.158114 years, and a reorder point of 12. Ordering and holding each cost 189.736660; relevant cost is 379.473319, and total cost is 6379.473319. Ordering 100 costs 6460: the model's annual saving is 80.526681, rather than the entire purchasing cost. The output document includes the comparison in

`knownQuantity` and `closedForm` exactness. In `prob_epq.json`, $p = 2400$ increases the optimal lot to 268.328157, with relevant cost 268.328157.

8.3 All-units quantity discounts

QSB evaluates each price tier and its feasible quantities, including purchasing, setup, and holding costs. The discount applies to every unit in the order; it is not a marginal block tariff. Holding cost is a cost per unit, rather than a percentage recalculated from the price. `prob_descuento.json` adds a 10% discount starting at 300 units. With `solve-inventory-json`, the selected quantity is $Q = 300$, with price 4.5, setup cost 120, holding cost 300, purchasing cost 5400, and total cost 5820. Relevant cost increases to 420, but the purchasing saving makes this tier preferable. The candidate table and `optimum` explain the decision; enumeration of the tiers is exact within the continuous model.

8.4 The newsvendor model

A single replenishment is chosen for a horizon with no later opportunity to replenish. For selling price v, purchasing cost c, salvage value s, and additional shortage penalty b, the marginal costs are $C_u = v - c + b$ and $C_o = c - s$. The critical fractile is

$$F(q^*) = \frac{C_u}{C_u + C_o}.$$

QSB supports normally distributed demand, calculating its quantile and expected sales, leftovers, shortages, and profit. The probability of covering all demand is the cycle service level; it is not the expected fraction of demand fulfilled. In `prob_newsboy.json`, mean 100, standard deviation 20, $v = 5, c = 2, s = 1, b = 1$, and zero fixed cost give critical fractile 0.8, order quantity 116.832425, expected sales 97.767247, leftovers 19.065178, shortages 2.232753, and profit 272.003808. The normal quantile uses approximate arithmetic; the `closedForm` label describes the method, rather than an exact decimal quantity or a guarantee for nonnormal demand.

8.5 Lot sizing

Ending inventory satisfies $I_t = I_{t-1} + q_t - d_t$. QSB minimizes

$$\sum_t \left(S_t \mathbf{1}_{q_t > 0} + c_t q_t + h_t \max(I_t, 0) + b_t \max(-I_t, 0) \right).$$

Dynamic programming uses integer demand and quantities, zero initial inventory, and zero final inventory, allowing backorders at a cost. It does not represent production capacity or configurable initial stock. Exactness refers to the bounded state space of this educational implementation: inventories between minus and plus total demand, and production in each period between zero and that demand. Computational cost grows with total demand and the horizon. `prob_lotes.json` has three demands of ten units, setup cost 20, unit production cost 2, holding cost 1, and backorder cost 10. QSB returns production $(20, 0, 10)$, ending inventory $(10, 0, 0)$, and cost 110. The policy $(10, 20, 0)$ also costs 110; an optimal result does not imply uniqueness. The decision table allows the balance and cost allocation to each period to be checked.

8.6 Review with uncertain demand

The four policies use normal demand per period and constant lead time. With independent demand increments, protection period T has mean DT and standard deviation $\sigma\sqrt{T}$. Safety stock is expressed using the normal quantile, and expected shortage cost uses the normal loss function. These are continuous-quantity approximations, marked `approximate`; they do not solve a general optimal policy for arbitrary distributions or random lead times.

The four examples share annual demand 1200, standard deviation 100, lead time 0.02 years, setup cost 30, holding cost 2, backorder cost 20, and purchasing cost 5. Every shortage is backordered; review cost is 5. The continuous order-up-to review file uses a supplied average order size of 200.

File under `manual/examples/`	Policy	Main result
`prob_qr.json`	Fixed quantity (Q, r)	$Q = 194.866241, r = 54.242691$
`prob_sS.json`	Continuous order-up-to	$r = 54.095104, S = 254.095104$
`prob_periodica.json`	Fixed interval	$T = 0.170783, S = 321.459989$
`prob_opcional.json`	Optional periodic	$T = 0.064550, s = 163.951339$

Optional review also returns $S = 353.687998$. Under fixed-quantity continuous review, safety stock is 30.242691, service level 0.983761, and relevant cost 450.217864. Under fixed-interval periodic review, protection includes the review interval plus lead time, 0.190783 years: confusing these intervals artificially reduces the stock required. Backorder and lost-sales fractions, their costs, and distributions are validated in the core.

8.7 macOS workflow and traceability

Open the normalized JSON or select an inventory variant in the model creator. Enter demand, costs, and the variant-specific parameters; validate before running. Keep units visible and review warnings. In the results, EOQ shows costs and a comparison with the supplied lot; discounts show candidates; the newsvendor model shows service and demand outcomes; lot sizing shows quantities and costs by period; stochastic review shows levels, safety stock, and costs. The analysis/JSON selector retains the core document for export without transcribing numbers from charts. Quantities and formulas come from QSBCore in both the CLI and macOS.

All files in this chapter are validated with `validate-inventory-json` and solved with `solve-inventory-json`; adding `-backend validate` validates without solving. The verifier `manual/verify_examples.py` checks the expected results in `manual/examples/manifest.json`; it requires no proprietary reference material.

8.8 Exercises

1. Compare integer orders of 189 and 190 in EOQ and explain why comparing costs is preferable to rounding blindly.

2. Double the lead time in the stochastic example: compare changes in protected demand and safety stock. Do both double?

3. Lower the backorder cost in the lot-sizing example and check whether negative inventories appear. Interpret backorders operationally.

Chapter 9

Scheduling and sequencing

9.1 Problem and assumptions

Scheduling assigns operations to machines and times. QSB minimizes the *makespan*, the time at which the last operation finishes [Pinedo, 2016]. Operations cannot be interrupted, a machine processes one operation at a time, and each job follows its route. Durations are integers in the specified unit. Setup times, working calendars, and random failures are not included. Due-date and weight fields are preserved, but the native objective remains makespan; it does not optimize weighted tardiness.

9.2 Flow shop: a common route

In a flow shop, all jobs visit the machines in the same order. The native solver searches for a permutation shared by all machines. For a given sequence, an operation starts when both the job and machine are available; its finish is its start plus its duration. Initial machine availability times are respected when provided.

In `manual/examples/plan_flowShop.json`, there are two jobs and two machines, initially available at zero:

Job	M1	M2
J1	2	3
J2	3	1

```
swift run qsb validate-scheduling-json manual/examples/plan_flowShop.json
swift run qsb solve-scheduling-json manual/examples/plan_flowShop.json --backend
    native
```

The sequence J1, J2 has makespan 6. On M1, J1 occupies $[0, 2]$ and J2 $[2, 5]$; on M2, J1 occupies $[2, 5]$ and J2 $[5, 6]$. The alternative J2, J1 finishes at 8. M2's initial idle time is not a missing operation: it is waiting for J1 to finish. The search is exact within the permutation model, with a limit of eight jobs and factorial cost; it does not prove optimality over all possible machine-specific sequences in a general flow shop.

9.3 Job shop: individual routes

In a job shop, each job may follow a different route. The example `plan_jobShop.json` requires J1: M1(2), M2(3), and J2: M2(2), M1(1). Precedence constraints follow from the order of each job's operations.

```
swift run qsb validate-scheduling-json manual/examples/plan_jobShop.json
swift run qsb solve-scheduling-json manual/examples/plan_jobShop.json --backend
    native
```

QSB obtains makespan 5: J1 uses M1 from 0 to 2 and M2 from 2 to 5; J2 uses M2 from 0 to 2 and M1 from 2 to 3. J1's total duration is 5 and provides a lower bound, so the schedule achieves the optimum. In this case, `jobSequence` reflects the operation dispatch order and may repeat names; it must not be interpreted as a permutation of jobs. Branch and bound with dominance pruning supports up to six jobs and 36 operations. These are educational limits: search time can grow substantially well before they are reached. Native is deterministic and marks both variants `fixtureScale`; Validate does not construct schedules, and External is unavailable.

9.4 Reading the Gantt chart and using macOS

Import the JSON envelope, whose `kind` is `flowShop` or `jobShop`. To enter or change operations, use the structured Definition form or the model JSON. These variants do not use the CPM/PERT activity editor. They are different problems, and the data for one do not replace those for the other. Validate and run with QSB Native. The solution opens Timeline: each row is a machine, bars are operations, and gaps show idle time. The horizontal scale lets you examine long schedules. Operation details identify the job, machine, start, finish, and duration; JSON also contains `idleBefore` and the schedules by machine.

Before accepting the result, check that no machine has overlapping bars, that each job retains its operation order, and that the maximum finish equals the makespan. A short schedule does not guarantee that due dates are met; compare them separately if they matter to the actual decision. The algorithms run in QSBCore, and the Gantt chart presents the same times exported by the CLI.

9.5 Exercises

1. Change J2's second operation in the flow shop from 1 to 5. Compare both permutations and verify the result.

2. In the job shop, add an initial availability time of 2 for M2. Draw the waiting times and check that the solver does not schedule operations before that time.

3. Set J1's due date to 4. Explain why changing this field does not turn the problem into tardiness minimization.

Chapter 10

Projects and production planning

10.1 CPM: precedence and the critical path

CPM operates on an acyclic activity network: an activity starts only after all its predecessors have finished. It does not allocate limited resources. The forward pass calculates early starts and finishes; the backward pass calculates late starts and finishes. Total float is late start minus early start. A critical activity has zero float.

In `manual/examples/plan_cpm.json`, A takes 3 days, B takes 2, and both precede C, which takes 4. Each activity has a normal cost of 10, a crash duration one unit shorter, and a crash cost of 15.

```
swift run qsb validate-project-json manual/examples/plan_cpm.json
swift run qsb solve-project-json manual/examples/plan_cpm.json --backend native
swift run qsb solve-cpm-crash-json manual/examples/plan_cpm.json 6 --backend native
```

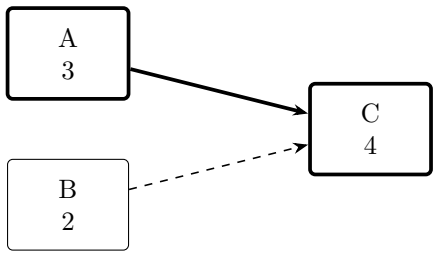


Figure 10.1: CPM activity network: the number below each name is its duration in days. Thick strokes identify critical activities and precedence; the dashed stroke shows the other precedence.

The normal project finishes in 7 days. A and C are critical; B has float 1. The total normal cost is 30. Crashing to a deadline of 6 reduces C from 4 to 3, with incremental cost 5 and total cost 35. Reducing A could also achieve the same cost; do not assume that the plan is unique.

Crash optimization assumes linear costs between normal and minimum durations, with slope $(C_{crash} - C_{normal})/(t_{normal} - t_{crash})$ when reduction is possible. It uses continuous times and an educational LP that considers paths; the number of paths can grow rapidly. It is not a policy for hiring resources and does not include indirect project costs. Deadlines shorter than what is achievable must be treated as infeasible, never as instructions to violate precedence constraints.

10.2 PERT: expected duration and uncertainty

PERT uses optimistic a, most likely m, and pessimistic b estimates, ordered $a \le m \le b$. The classical approximation calculates

$$E[T] = (a + 4m + b)/6, \qquad \mathrm{Var}(T) = ((b - a)/6)^2.$$

In `plan_pert.json`, the triples are A(2,3,4), B(1,2,3), and C(3,4,5), with the same precedence relationships as in the CPM example.

```
swift run qsb solve-project-json manual/examples/plan_pert.json --backend native
```

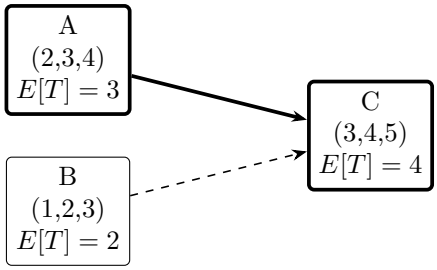


Figure 10.2: PERT network: each activity shows its optimistic, most likely, and pessimistic triple and its expected duration in days. Thick strokes identify the critical path calculated using those means.

Scheduling with expected durations again gives a duration of 7, with A and C critical. The reported variance is 2/9 and the standard deviation is 0.471405 days. In this example, there is a single critical path, and summing its variances assumes independence. The implementation sums the variances of all activities marked critical; if there are multiple critical paths, that total is neither the variance of a single path nor that of the maximum over them. The critical path may change between realizations; replacing random durations with their means does not calculate the expected maximum exactly. The `exact` label describes the calculation using those expected durations, not an exact probabilistic prediction of the completion date. QSB does not return a probability of meeting a deadline or simulate that uncertainty in this module.

10.2.1 CPM and PERT in macOS

Import the project envelope or create a CPM/PERT definition. The activity editor collects names, predecessors, and durations; CPM adds costs, and PERT adds the three estimates. Missing predecessors and cycles appear in Validation. Switching methods preserves data for both methods after initialization: check the estimates when returning to PERT. Copy Table/Paste Table lets you transfer activities from a spreadsheet. Run produces duration, critical activities, early/late bars, float values, and a timing table. Review JSON for the variance and exact values. The CLI crash operation is a separate workflow; the normal view does not imply that a deadline has been optimized. Native and Validate are available; External does not currently solve project models.

10.3 Aggregate planning

Aggregate planning balances demand across periods through regular production, overtime, subcontracting, inventory, and backorders. When capacity depends on workforce, it adds hiring and layoffs. A basic balance is

$$I_t - B_t = I_{t-1} - B_{t-1} + R_t + O_t + S_t - D_t.$$

The objective sums production, idle capacity, workforce, inventory, and backorder costs according to the data. QSB normalizes the historical simple, transportation, and LP methods into a single continuous LP formulation; the `method` field preserves the input identity rather than selecting three different native algorithms.

The example `plan_aggregate.json` has two months, demand 4 and 6, regular capacity 5 each month, regular cost 2 per unit, overtime capacity up to 2 at cost 5, inventory cost 1, and no backorders or subcontracting. It does not depend on workforce and starts with no inventory.

```
swift run qsb validate-aggregate-json manual/examples/plan_aggregate.json
swift run qsb solve-aggregate-json manual/examples/plan_aggregate.json --backend
    native
```

The plan produces 5 in each month; it holds 1 at the end of the first and zero at the end of the second. It costs $10 \cdot 2 + 1 = 21$. Producing exactly the first month's demand and using one overtime unit in the second would cost $9 \cdot 2 + 5 = 23$. Units must be consistent: capacity per worker is not total capacity, and `capacityRequirementPerUnit` converts production into capacity. Vectors must have the same number of periods; `null` in optional maximum fields means no upper bound. Workforce and quantities are continuous, even when the business needs whole people. Do not round a solution without rechecking balances and capacity.

In macOS, the period editor separates demand, costs, inventory/sources, and workforce. Copy Table transfers all 20 period columns; Paste Table validates the range before replacing it. Run shows stacked production and demand, inventory, backorders, and a plan table. A backorder appears below zero in the chart, but its model quantity is nonnegative. Native solves the educational LP; External uses HiGHS for the same formulation if installed. Validate does not produce a plan. The optimal objective refers to the costs and bounds entered, not to omitted criteria such as workforce stability and its social implications.

10.4 MRP: requirements explosion

MRP starts from a master production schedule (MPS), a bill of materials (BOM), inventory, scheduled receipts, and lead times. It processes parents before components: a parent's order release generates gross requirements for its components, multiplied by the quantity used per unit [Silver et al., 1998]. For an item with safety stock SS, gross requirement G_t, and scheduled receipt SR_t, net requirement is $\max(0, G_t + SS - I_{t-1} - SR_t)$. A planned receipt covers that requirement, and its release is shifted backward by the lead time.

`plan_mrp.json` has weeks S0–S3. Five units of P must be received in S3; P uses two units of C per unit. Both have a one-week lead time, zero initial inventory and safety stock, the LFL rule, and unlimited capacity.

```
swift run qsb validate-mrp-json manual/examples/plan_mrp.json
swift run qsb solve-mrp-json manual/examples/plan_mrp.json --backend native
```

P receives 5 in S3 and releases 5 in S2. C needs 10 in S2, receives 10 then, and releases 10 in S1. Projected inventories remain zero. C's requirements come from P's release, not from the final demand date. The BOM must be acyclic, and all its identifiers must exist.

The available rules are LFL, EOQ, LUC, LTC, and PPB. LFL receives the net requirement; EOQ starts from $\sqrt{2DS/H}$ and receives at least the current requirement. In EOQ, `lotSizingLookahead` is interpreted as a lot multiple $m = \max(1, \text{configured value})$, not as a number of periods: the quantity is $\max(\text{requirement}, m\lceil EOQ/m \rceil)$. If annual holding cost is zero, it receives the net requirement. LUC compares unit cost; LTC and PPB currently use the same selector, which minimizes the absolute difference between holding cost and ordering cost within the configured horizon. These are forward-looking rules, not a global optimization

of multilevel costs. QSB labels the MRP document `heuristic`, even when the particular case uses LFL. Finite capacity generates `capacityExcess`; it does not level or reschedule orders. If a release falls before the horizon, QSB accumulates it in the first bucket using $\max(0, t - L)$; that record requires operational review and does not prove that the lead time can be met.

In macOS, Item Master collects parameters, Schedules collects quantities by period, and Bill of Materials collects parent/component relationships. Select an item to edit or read its schedule; long horizons are grouped for navigation. Copy/Paste Table always exchanges the complete range for its section. The solution shows requirements, inventory, receipts, releases, and capacity excesses by item, with JSON as an audit record. Native and Validate are available; External is not implemented for MRP.

10.5 Exercises

1. In CPM, increase B from 2 to 4. Recalculate the critical path and float.
2. In PERT, increase only A's pessimistic estimate. Compare duration and variance; explain which assumptions limit a probabilistic interpretation.
3. Raise aggregate inventory cost from 1 to 4. Compare early production with overtime.
4. Add initial inventory of 4 units of C in MRP and check its net requirement. Then limit C's capacity in S2 to 5 and explain the excess.

Chapter 11

Facilities and line balancing

11.1 Locating a facility

Continuous location minimizes weighted distance to existing points:

$$\min_{x,y} \sum_i w_i d((x,y),(x_i,y_i)).$$

Weights represent interactions multiplied by unit cost, not probabilities. QSB supports one new facility and minimization; it does not solve discrete warehouse selection, multiple new facilities, or problems with geographic barriers.

The distance measure determines the algorithm. For rectilinear distance, $d = |x - x_i| + |y - y_i|$, weighted medians are used coordinate by coordinate. For squared Euclidean distance, $d = (x - x_i)^2 + (y - y_i)^2$, the weighted centroid is used. For ordinary Euclidean distance, Weiszfeld iteration is used and the result is approximate. Do not change the measure merely to obtain visually more attractive coordinates: it changes the objective and its units.

`manual/examples/plan_location.json` places F1 at $(0, 0)$ with weight 1 and F2 at $(4, 0)$ with weight 3, and uses squared Euclidean distance. The directed matrix contains these values from the existing facilities to the new one; there are no other interactions. The effective weight sums both interaction directions between each existing point and the new one: recording the same flow in both directions doubles the weight.

```
swift run qsb validate-facilities-json manual/examples/plan_location.json
swift run qsb solve-facilities-json manual/examples/plan_location.json --backend
    native
```

The center is $(3, 0)$ and the objective is $1 \cdot 9 + 3 \cdot 1 = 12$. In macOS, Definition lets you edit roles, IDs, coordinates, distance, and the matrix. New coordinates may be left blank; existing coordinates are required. IDs are interpreted within the existing/new role, so both can use ID 1. The solution displays points and weighted connections and details each contribution. Use JSON to preserve the complete matrix.

11.2 Balancing a line

With indivisible tasks of duration t_i, precedence constraints, and fixed cycle time C, the objective minimizes the number of stations. A station cannot exceed the cycle time, and a predecessor's station cannot follow its successor's station; predecessor and successor tasks may share a station if performed in order. The lower bound $\lceil \sum_i t_i / C \rceil$ ignores incompatibilities and precedence, so it is not always attainable.

`plan_line.json` contains A(2), B(3), C(4), with A and B preceding C, and a cycle time of 5 minutes.

```
swift run qsb validate-facilities-json manual/examples/plan_line.json
swift run qsb solve-facilities-json manual/examples/plan_line.json --backend native
```

A and B are assigned to station 1 and C to station 2. There are 9 minutes of work and 10 minutes of capacity per cycle: efficiency is $9/(2 \cdot 5) = 0.9$ and balance delay is $1 - 0.9 = 0.1$. Idle time is 0 and 1 minute, respectively. The lower bound is 2 and is attained, proving optimality for the model. Exact bitmask dynamic programming supports at most 24 tasks; a task longer than the cycle time makes the problem infeasible. The `isolated` field is preserved for compatibility, but the current native solver does not require a separate station for marked tasks. Do not use it as an effective safety or incompatibility constraint.

The macOS editor supports tasks, durations, immediate successors, and flags; Copy/Paste Table transfers the table. Deleting a referenced task leaves the reference visible so that it can be repaired. The solution shows station workload and idle time, assignments, precedence context, efficiency, and balance delay. Efficiency does not measure total batch production time or include breakdowns or product changeovers.

11.3 Facility layout

Layout evaluates departments positioned on a grid. Each department occupies the union of one or more cell rectangles; a repeated cell within the same department is counted once. Rectangle coordinates are inclusive and start at 1. The load–distance objective uses centroids:

$$Z = \sum_{i,j} w_{ij}(|r_i - r_j| + |c_i - c_j|).$$

The matrix is directed: entering the same weight in both directions doubles the contribution. The centroid approximates physical travel; it does not model doors, mandatory aisles, or obstacles.

The example `plan_layout.json` has one row and three cells A, B, C, all with area one and movable. Its only weights are A–B=1, A–C=10, and B–C=1.

```
swift run qsb validate-facilities-json manual/examples/plan_layout.json
swift run qsb solve-facilities-json manual/examples/plan_layout.json --backend
    native
swift run qsb solve-facilities-json manual/examples/plan_layout.json \
  --backend native --layout-strategy pairwise-swap
```

Initial evaluation gives $1 + 20 + 1 = 22$ and does not change the arrangement. Pairwise swaps A and B: B, A, C costs $1 + 10 + 2 = 13$, an improvement of 9. The search swaps only movable departments of equal area when doing so improves the objective. It terminates at a local optimum in that neighborhood; it does not guarantee a global optimum or automatically construct arbitrary geometry. Fixed areas are not swapped. Overlaps between movable areas are errors; overlaps involving fixed areas may produce warnings and must be reviewed.

In Definition, you edit the grid, departments, fixed flags, rectangles, and flow/cost matrix. Select a department to access its rectangles; the transferred table includes IDs for all of them. In large matrices, sources and destination groups are selected, although Copy/Paste Table transfers the complete matrix. Choose Initial or Pairwise before Run. The Layout view lets you adjust scale, grid, and flows, and examine centroids, swaps, and contributions. For large grids, the overview draws fewer lines; exact ranges remain available in details and JSON.

11.4 Backend, validation, and decision

All three families support Native and Validate, with no operational External backend. Validate reports invalid data without calculating a location, layout, or line balance. Documents preserve algorithm and exactness: centroid/median location is closed-form, Weiszfeld is approximate, line balancing is exact at educational scale, and Pairwise is heuristic. Evaluating an initial layout determines only its cost, not its optimality. The examples use the facilities envelope with `kind` equal to `location`, `lineBalancing`, or `layout`. Do not confuse that envelope with the individual JSON formats accepted by commands specific to each subfamily. Validate the same file you will solve and export the model and solution to retain a traceable record of accepted data.

11.5 Exercises

1. Change location to rectilinear distance. Explain the new coordinate using weighted medians and the new units of the objective.

2. Reduce the line's cycle time to 4. Determine the minimum number of stations and check the impact on efficiency.

3. Fix A in the layout. Solve with Initial and Pairwise and compare permitted moves. Explain why a search with no improvement does not prove optimality over all possible layouts.

Part IV

Uncertainty and quality

Chapter 12

Queuing models

12.1 From flow to waiting

Queuing theory relates variability, capacity, and congestion [Gross et al., 2008]. In Kendall notation, M denotes Poisson arrivals or exponential times, depending on its position. Service rate μ is per server; mean service time is $1/\mu$. All rates share a time unit. QSB implements M/M/1 and a finite queue with multiple servers using a birth-and-death model.

12.2 M/M/1 in steady state

The assumptions are independent arrivals, exponential service, one server, and unlimited population and waiting space. Stability requires $\lambda < \mu$. With $\rho = \lambda/\mu$:

$$P_0 = 1 - \rho, \quad L = \frac{\rho}{1-\rho}, \quad L_q = \frac{\rho^2}{1-\rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\lambda}{\mu(\mu - \lambda)}.$$

L includes customers being served; L_q includes only those waiting. Little's law gives $L = \lambda W$ and $L_q = \lambda W_q$. The backend reports `mm1ClosedForm`/`closedForm`. It does not calculate the distribution or percentiles of an individual waiting time.

12.2.1 Example: a service counter

`manual/examples/prob_cola.json` specifies four arrivals/hour and six services/hour. Run from the root:

```
.build/debug/qsb validate-queuing-json manual/examples/prob_cola.json
.build/debug/qsb solve-queuing-json manual/examples/prob_cola.json
```

Utilization is 2/3, $P_0 = 1/3$, $L = 2$, $L_q = 4/3$, $W = 0.5$ hours, and $W_q = 1/3$ hours: thirty minutes in the system, twenty waiting, and ten receiving service. Being busy two thirds of the time does not mean that two thirds of customers wait exactly twenty minutes. If arrivals reach six/hour, the model no longer has these steady-state means, and QSB returns a stability diagnostic.

12.3 Multiple servers and finite capacity

With c servers and b waiting places, system capacity is $K = c + b$. State n has arrival rate λ when $n < K$, and zero when full; its departure rate is $\min(n, c)\mu$. The probabilities are

normalized after being obtained from

$$p_n = p_0 \prod_{i=1}^{n} \frac{\lambda}{\min(i, c)\mu}.$$

Blocking probability is p_K, the effective arrival rate is $\lambda_e = \lambda(1 - p_K)$, and Little's law uses λ_e, rather than the offered rate. Because it is finite, this model can have a stationary distribution even when offered demand exceeds its service capacity; this produces blocking, rather than extra capacity.

QSB requires exponential arrivals in batches of one and an infinite customer population. It uses mean service time to obtain the rate per server; nonexponential labels produce warnings because their variability is not modeled. The document always retains `approximate` for this route. A case with normal service times must not be presented as an exact M/G/c/K solution.

12.3.1 Example: two counters and three waiting places

`prob_cola_finita.json` retains four arrivals/hour and a mean service time of ten minutes, with two servers and three waiting places. Validate and solve using the same commands with the file changed. The results are:

Metric	Value
Total capacity	5
Blocking probability	0.004124
Effective arrivals per hour	3.983505
Average utilization per server	0.331959
Mean number in the system	0.738144
Mean number waiting	0.074227
Time in the system (hours)	0.185300
Waiting time (hours)	0.018634

Mean waiting time is approximately 1.118012 minutes among admitted customers. The total utilization of both servers corresponds to 0.663918 busy servers, rather than the individual utilization percentage.

12.4 Costs, interface, and limitations

Optional costs distinguish busy and idle servers, waiting customers, and customers in service; the finite queue adds rejection and waiting places. State costs are weighted by expected occupancy; rejection cost is multiplied by the rate of blocked customers. The total is a cost per time unit, rather than per customer.

In macOS, the creator offers both variants. Enter rates for M/M/1, or mean times, servers, and waiting space for a finite queue; validate and solve. The metrics panel shows utilization, effective flow, blocking, numbers, and times. The finite variant also shows state probabilities, highlighting the full state; optional costs are itemized. Check the time unit and assumptions alongside the results. Normalized JSON is the alternative for inspection and export.

Use `solve-queuing-json` to accept normalized input.

The historical commands `solve-mm1-json` and `solve-finite-queue-json` accept a legacy file despite their suffix: the suffix indicates JSON output. Always use the generic route to exchange normalized inputs. Native and validation backends are available; no external solver is installed for this family.

Priorities, customer abandonment due to impatience, finite customer populations, general batch arrivals, and rate changes by time of day are not modeled. The simulation in the Markov and simulation chapter allows a different class of assumptions to be explored, but its estimates do not replace these steady-state formulas.

12.5 Exercises

1. Change λ to 5.5 in M/M/1 and compare utilization and waiting. Explain why the growth in waiting time is nonlinear.

2. Remove the waiting places from the finite queue. Check that blocking occurs without a positive queue length, and use λ_e to verify Little's law.

3. Compare adding a server with adding five waiting places: which changes service capacity, and which changes the space for customers?

Chapter 13

Forecasting and regression

13.1 A forecast is an operational hypothesis

Forecasts inform inventory, capacity, and planning. A straight line that fits the past well does not guarantee future accuracy [Hyndman and Athanasopoulos, 2021]. QSB implements linear trend, moving average, simple exponential smoothing, multiplicative seasonal decomposition, and least-squares linear regression. The JSON is a request: it contains the model, method, horizon, and parameters. This preserves the chosen method alongside the data. Native and validation backends are available; external integration is a future extension.

13.2 Data and errors

Observations are processed in row order; labels do not reorder dates or fill missing periods. A monthly series must have consistent monthly spacing. QSB returns fitted values, residuals $e_t = y_t - \hat{y}_t$, MAD, MSE, and MAPE:

$$\text{MAD} = \frac{1}{m}\sum |e_t|, \quad \text{MSE} = \frac{1}{m}\sum e_t^2, \quad \text{MAPE} = \frac{100}{m_+}\sum_{|y_t|>10^{-12}} |e_t/y_t|.$$

m counts the available fitted rows, rather than necessarily the entire history; m_+ excludes actual values whose magnitude is no greater than 10^{-12}. If all are excluded, MAPE is unavailable. MAD retains the series' units; MSE has squared units; MAPE is reported as a percentage. Comparing methods requires considering that moving average and smoothing omit different initial rows.

13.3 Linear trend: a complete example

The model $y_t = a + bt + e_t$ estimates a, b by least squares with indices $t = 1, \ldots, n$. `manual/examples/prob_pronostico.json` contains $(12, 14, 16, 18, 20, 22)$, month as its unit, and a horizon of two. From the root:

```
.build/debug/qsb validate-forecast-json manual/examples/prob_pronostico.json
.build/debug/qsb solve-forecast-json manual/examples/prob_pronostico.json
```

QSB obtains $a = 10, b = 2$, forecasts of 24 and 26, and zero for all three error measures. These artificial data were chosen to check the formula and indices: zero error is not evidence about real demand. The result retains the request, the `linearTrend` algorithm, and `exact` exactness, which describes the method's calculation rather than exact knowledge of the future.

13.4 Moving average and smoothing

For window k, the one-step forecast is

$$\hat{y}_{t+1} = \frac{1}{k}\sum_{j=0}^{k-1} y_{t-j}.$$

QSB extends horizons by using earlier forecasts within the window when actual values run out. The file `prob_movil.json` selects a window of three for the same series. The results are 20 and 20.666667: the second average uses $(20, 22, 20)$. MAD is 4, MSE 16, and MAPE 20.134680%, calculated over the final three rows. This lag is expected when the series has a trend.

Simple smoothing applies

$$F_{t+1} = \alpha y_t + (1-\alpha)F_t, \qquad 0 < \alpha \leq 1.$$

The initial forecast is set to the first observation. In `prob_suavizado.json`, $\alpha = 0.3$ produces 18.117660 in both future periods. The method has no trend component; it does not automatically extrapolate growth. Its MAD is 4.078440, MSE 18.233818, and MAPE 22.081810%. A large alpha reacts quickly and may follow more noise; a small one smooths and may lag. Both examples run with `solve-forecast-json` and are validated with `validate-forecast-json`.

13.5 Multiplicative seasonality

The method combines the estimated trend with seasonal factors: $\hat{y}_t = (a + bt)s_j$. QSB calculates actual/trend ratios, averages them by seasonal position, and normalizes the factors to have a mean of one. This is an educational decomposition, rather than Holt–Winters or an ARIMA model. Seasonal length and observation order must correspond to the phenomenon; at least two complete cycles and values/trend compatible with multiplicative division are required. `prob_estacional.json` uses length two for the previous linear series. Both factors are one; the forecast is again $24, 26$. The case checks that specifying a season does not invent oscillations when data coincide with the line. To study actual seasonality, replace the series with several oscillating cycles and review residuals before interpreting the factors.

13.6 Explanatory regression

The model is $y_i = \beta_0 + \sum_j \beta_j x_{ij} + e_i$, with an intercept. Predictors must have linearly independent columns; a constant column duplicates the intercept. The core detects rank deficiency. `prob_regresion.json` contains activity $1, 2, 3, 4, 5$ and consumption $5, 7, 9, 11, 13$. With `solve-forecast-json`, QSB returns intercept 3, activity coefficient 2, zero SSE, and $R^2 = 1$. The request contract's horizon must also be positive for regression, although its output fits the supplied observations and does not generate future predictor rows. For a new activity level, the estimated equation allows a prediction to be calculated, without QSB reporting an interval for it.

A coefficient is a conditional association, rather than causality. R^2 measures within-sample fit; it does not identify overfitting or guarantee extrapolation. This workflow does not report coefficient standard errors, hypothesis tests, or prediction intervals. Small original datasets allow the contract to be checked, but do not assess generalization ability.

13.7 Working in macOS and evaluation

In the forecasting editor, select the method, parameters, and horizon; enter historical rows in temporal order. Values can be pasted into one column, or labels and tab-separated values into two columns. Paste validation is atomic: a malformed row does not partially replace the history. Regression uses variable names and explanatory rows. Validate, solve, and review actual, fitted, and future curves; these curves and the residual chart share observation order. Visibility and horizontal-scale controls help with long series. The summary shows parameters and metrics; the JSON selector preserves the request and result. Applying JSON updates the draft only when decoding succeeds.

To evaluate an operational forecast, reserve the final periods, fit using the earlier history, and compare against the held-out data. The errors QSB reports are the fits from the executed calculation; they do not automatically constitute an out-of-sample test. Do not select parameters by repeatedly examining the same held-out set. Document units, window, alpha, season, and horizon when delivering the analysis.

13.8 Exercises

1. Compare windows of two and four for the growing series. Explain the lag and check which rows contribute to MAD.

2. Replace one observation with zero and check the definition of MAPE. Compare its suitability with MAD.

3. Enter a second predictor equal to twice the first and observe the rank diagnostic. Explain why more columns do not always add information.

Chapter 14

Markov processes and simulation

14.1 Two ways to describe uncertainty

A Markov chain propagates probabilities among states; a simulation constructs realizations of events and estimates measures through repetition. The former can yield a stationary distribution through equations; the latter supports more detailed networks and rules, at the cost of sampling error. QSB maintains separate normalized models and results for the two procedures [Ross, 2019, Law, 2015].

14.2 Finite chains and the row convention

P_{ij} is the probability of moving from state i to j in one period. Each row sums to one. The distribution is a row vector:

$$s_{t+1} = s_t P, \qquad \pi P = \pi, \qquad \sum_i \pi_i = 1.$$

Transitions are assumed time homogeneous, and the next state depends on the current one without additional information from the past. Costs c_i are per state per period; their expectation is $s_t c$. They are not transition costs.

14.2.1 Example: machine availability

`manual/examples/prob_markov.json` defines Operativa (operational) and Parada (stopped), costs $(0, 100)$, initial distribution $(1, 0)$, a horizon of three, and

$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.4 & 0.6 \end{pmatrix}.$$

From the repository root:

```
.build/debug/qsb validate-markov-json manual/examples/prob_markov.json
.build/debug/qsb solve-markov-json manual/examples/prob_markov.json
```

The results are:

Period	Operativa	Parada	Expected cost
0	1	0	0
1	0.9	0.1	10
2	0.85	0.15	15
3	0.825	0.175	17.5

The stationary distribution is $(0.8, 0.2)$, with expected cost 20 per period. This is a long-run mean, rather than a cumulative cost over three periods. This chain is irreducible and aperiodic, so its transient distributions converge to it. In general, stationary uniqueness and convergence from every initial distribution are not equivalent: a periodic chain may oscillate.

QSB solves a stationary linear system and propagates matrices; it reports `exact` as the algorithmic character, using floating-point arithmetic. The method requires a unique stationary distribution and returns an error if its system is singular. It does not choose an arbitrary mixture for chains with several closed classes. When no initial distribution is supplied, it does not invent one: transient output is empty. Validating rows and dimensions does not guarantee that the stationary system is solvable.

In macOS, the editor supports names, costs, optional initial probabilities, the transition matrix, and the horizon. Check row orientation before solving; the result shows the stationary distribution, propagation, costs, and assumptions. Editing states keeps dimensions consistent; invalid text remains visible until corrected. JSON remains available for interchange.

14.3 Discrete-event simulation

A network contains entity sources, queues, servers, and gates. Routes include probabilities and transfer times; service rules may depend on entity type. QSB supports constant, exponential, normal, uniform, and triangular distributions. Negative normal samples are clipped to zero; they are not resampled. This clipping changes the mean and variability relative to an unclipped normal distribution. Parameters represent distribution times or values, rather than rates per time unit. For example, an exponential mean of 0.25 hours corresponds to four arrivals/hour.

The educational backend models finite FIFO queues, alternative servers, probabilistic routing, transfers, and assembly synchronization. Its outputs are `approximate` estimates, even when a fixed seed makes them reproducible. There is no warm-up deletion, interactive event trace, or guarantee of steady state. No external simulation backend is available either.

14.3.1 Example: a simulated service counter

`prob_simulacion.json` connects source Cliente (customer), queue Espera (waiting) with twenty places, and server Atención (service). Each arrival brings one entity, interarrival time is exponential with mean 0.25 hours, and mean exponential service time is 1/6 hours. Transfers take zero time.

```
.build/debug/qsb validate-simulation-json manual/examples/prob_simulacion.json
.build/debug/qsb solve-simulation-json manual/examples/prob_simulacion.json
.build/debug/qsb solve-simulation-json manual/examples/prob_simulacion.json --
    replications 3
```

The default run lasts 1000 hours, with seed 1, and no warm-up removed. It generates 4005 entities, completes 4004, rejects none, and estimates mean queue length 1.202238, maximum 15, and utilization 0.664470. The uncompleted customer may be in the queue or in service at termination: it is not necessarily rejected.

With three independently seeded replications, mean queue length is 1.229240, with approximate normal 95% interval $[1.146668, 1.311813]$; utilization is 0.659950, with interval $[0.638179, 0.681722]$. Reported aggregate counters are rounded means: 4006 generated and 4005 completed, rather than the sum of three runs. Maxima, however, retain the maximum across replications. The base seed, horizon, and number of replications appear in the output. The CLI exposes replications but no options for choosing a seed or horizon in these commands; those values come from the core's default options.

The interval formula uses $\bar{x} \pm 1.96 s/\sqrt{R}$. With few replications, the normal approximation may be poor; increasing R reduces sampling error, but does not correct startup bias or an incorrect distribution. Do not interpret the interval as a guarantee that the model describes reality. The analytical M/M/1 in the queuing chapter gives $L_q = 4/3$ and $\rho = 2/3$. The simulation has a finite queue and finite horizon: closeness is useful for checking units, but does not require exact equality. Three replications do not justify inferring a failure because an interval does not contain the analytical value, either.

14.4 macOS workflow

In the simulation definition, complete `Components`, `Routes`, and `Service Rules`. Enter the parameters of each distribution separately. Names must match route references and entity types. Complete tables can be exchanged with spreadsheets; pasting checks each section before replacing it. In the Run panel, select between one and one hundred replications, validate, and run. The interface shows computation status and prevents duplicate runs; changing the model invalidates a pending result. This does not stop a calculation already started in the core.

The results panel shows entities generated/completed, completion rate, horizon, seed, queue, and utilization, with intervals when available. Consult tables and JSON as well as charts. Keep the network, rules, seed, horizon, and replications when comparing scenarios: changing any of these may change the interpretation of the difference.

14.5 Exercises

1. Remove the initial distribution from the Markov case and check that only stationary analysis remains. Then change the matrix to the identity and observe the singular-system error.

2. Double the simulated mean service time and compare completions, rejections, and utilization. Explain why queue capacity is not equivalent to service capacity.

3. Repeat the case with ten replications and compare interval widths. Distinguish variation between runs from differences between model assumptions.

Chapter 15

Quality control and acceptance sampling

15.1 Monitoring a process or accepting a lot

Statistical control looks for signals of change in a process; acceptance sampling makes decisions about lots using samples. A stable process can produce defects, and accepting a lot does not certify that all its units conform. Both procedures require consistent definitions and sampling [Montgomery, 2020].

QSB supports c, p, and $\bar{X}$-R charts, Pareto charts, and normal probability plots; it also evaluates single and double binomial acceptance plans. The native backend is deterministic, and the validation backend does not solve. No external backend is available for these families.

15.2 c and p charts

The c chart counts defects in an inspection unit of constant size. An item may have more than one defect. Under a Poisson count assumption,

$$CL = \bar{c}, \qquad LCL = \max(0, \bar{c} - 3\sqrt{\bar{c}}), \quad UCL = \bar{c} + 3\sqrt{\bar{c}}.$$

`manual/examples/prob_calidad.json` contains counts $(2, 3, 4, 3, 2, 4, 3, 3, 2, 4)$. From the root:

```
.build/debug/qsb validate-quality-json manual/examples/prob_calidad.json
.build/debug/qsb solve-quality-json manual/examples/prob_calidad.json
```

The center line is 3, the lower limit zero, and the upper limit 8.196152. There are no points outside the limits or signals from the implemented rules. This is an educational check of the formula; ten counts are insufficient to establish an industrial process as stable. Inspection units of varying sizes require another approach; QSB does not implement a u chart in this route.

The p chart uses the proportion of defective units, with size n_i for each subgroup. Its weighted center is

$$\bar{p} = \frac{\sum_i n_i p_i}{\sum_i n_i}, \qquad CL_i = \bar{p}, \quad CL_i \pm 3\sqrt{\bar{p}(1-\bar{p})/n_i},$$

with limits clipped to $[0, 1]$.

In `prob_p.json`, sizes $(100, 100, 200)$ and proportions $(0.02, 0.04, 0.03)$ produce $\bar{p} = 0.03$; upper limits are 0.081176 for the first two subgroups and 0.066187 for the third. The lower

limit is zero. Input uses fractions: 0.03 means 3%, rather than 0.03%. Three-sigma limits are a statistical approximation to the data distribution, even though QSB labels the application of the formulas as `exact`.

15.3 Mean and range charts

Rational subgroups combine measurements taken under comparable conditions. With $\bar{\bar{X}}$ as the mean of subgroup means and $\bar{R}$ as the mean of ranges,

$$LCL_{\bar{X}}, UCL_{\bar{X}} = \bar{\bar{X}} \mp A_2\bar{R}, \qquad LCL_R = D_3\bar{R}, \quad UCL_R = D_4\bar{R}.$$

QSB uses standard constants for uniform sizes between two and ten. Review variation in R before interpreting shifts in means. `prob_xr.json` contains subgroups $(9, 10, 11)$, $(10, 11, 12)$, and $(8, 9, 10)$. The results are $\bar{\bar{X}} = 10$, $\bar{R} = 2$, mean limits $[7.954, 12.046]$, and range limits $[0, 5.148]$, with no points outside the limits. Control limits describe process variability; they are not design tolerances. QSB does not calculate Cp/Cpk indices in this workflow.

15.4 Pareto and normal probability

Pareto aggregates categories, sorts them by descending frequency, and reports shares and cumulative shares. `prob_pareto.json` has rows $(4, 2, 1)$ and $(2, 1, 0)$ for Rayas (scratches), Manchas (stains), and Roturas (breakages). Their totals are $(6, 3, 1)$, the overall total is 10, shares are $(0.6, 0.3, 0.1)$, and cumulative shares are $(0.6, 0.9, 1)$. Fields named `percentage` in JSON store fractions, rather than values from zero to one hundred. Prioritizing Rayas by frequency does not establish that it is the category with the greatest cost or risk.

The normal plot sorts measurements and assigns Blom plotting positions $p_i = (i-3/8)/(n+1/4)$; it uses $z_i = \Phi^{-1}(p_i)$, fitting measurement against normal score. `prob_normal.json` contains $(8, 9, 10, 11, 12)$; mean 10, sample standard deviation 1.581139, intercept 10, slope 1.742919, and correlation 0.997902. It is labeled `approximate` because of the approximate normal quantile. High correlation with five points does not demonstrate normality; this is not a test with a p-value. These four examples run using the same quality commands with the file changed.

15.5 Signals and the quality interface

QSB flags points beyond three sigma and fixed Western Electric rules: two of three beyond two sigma on the same side, four of five beyond one sigma on the same side, and eight on the same side of the center. Points already flagged beyond three sigma are excluded from the counts for the two-of-three and four-of-five zone rules. There is no rule configuration or normalized metadata for cause, action, and comment. Signal indices identify observations to investigate, rather than automatic cause diagnoses. Clipped attribute-chart limits affect the scale used by this implementation's rules: retain the output and review each signal alongside the chart.

In macOS, select the variant in the native editor and complete its table: counts/proportions, subgroups, categories, or measurements. Changing variants preserves inactive tables during the session, but JSON exports only the active one. Paste complete tables to replace them atomically; incomplete imported data remain as invalid cells. After validation and solving, charts show centers, limits, and signals; Pareto shows frequency and cumulative share; the normal plot shows points and a fitted line. Consult tables and JSON for the precise values of the calculation, and preserve acquisition order.

15.6 Single acceptance plan

For sample size n and maximum acceptable number of defective units c, the lot is accepted if $X \le c$. Under independent binomial sampling with proportion p, the operating characteristic curve is

$$P_a(p) = \sum_{x=0}^{c} \binom{n}{x} p^x (1-p)^{n-x}.$$

AQL is the acceptable quality level and RQL the rejectable quality level. Actual producer's risk is $\alpha = 1 - P_a(AQL)$; consumer's risk is $\beta = P_a(RQL)$. Entering nominal risks does not alter an already supplied plan to satisfy them.

`prob_muestreo.json` sets $n = 20, c = 1, AQL = 0.01, RQL = 0.15$, nominal risks $0.05, 0.10$, and lot size 500.

```
.build/debug/qsb validate-acceptance-json manual/examples/prob_muestreo.json
.build/debug/qsb solve-acceptance-json manual/examples/prob_muestreo.json
```

QSB returns $P_a(AQL) = 0.983141$, producer's risk 0.016859, and consumer's risk 0.175558. The plan exceeds the nominal consumer's risk: it must not be described as satisfying $\beta = 0.10$. Output contains 101 curve points from 0 to 100% defective, as well as measures at AQL and RQL. This grid does not replace continuous evaluation when optimizing a risk between points.

Average sample number ASN is 20. Under rectifying inspection of rejected lots and removal of inspected defective units,

$$ATI = n + (N-n)(1-P_a), \qquad AOQ = pP_a(N-n)/N.$$

At AQL, ATI is 28.092482 and AOQ 0.009438, that is 0.943815%. If rejected lots are returned without full inspection, these measures do not describe the actual procedure. QSB evaluates binomial probabilities exactly within the assumption, but sampling a large fraction of the lot without replacement requires considering the hypergeometric distribution; the core provides that probability, rather than a selector in this command.

15.7 Double plans and design

In a double plan, n_1 units are inspected. If the number of defective units is $x_1 \le c_1$, the lot is accepted; if $x_1 \ge r_1$, it is rejected; in between, n_2 *additional* units are inspected. Final acceptance occurs if $x_1 + x_2 \le c_2$. Confusing n_2 with cumulative sample size changes the plan. Its acceptance probability is

$$P_a = P(X_1 \le c_1) + \sum_{x=c_1+1}^{r_1-1} P(X_1 = x) P(X_2 \le c_2 - x).$$

The file `prob_doble.json` uses $n_1 = 10, c_1 = 0, r_1 = 3$, $n_2 = 10, c_2 = 1$, with the same quality levels and lot size as before. Changing the file in `solve-acceptance-json` gives producer's risk 0.013001, consumer's risk 0.265274, ASN at AQL 10.955041, and ASN at RQL 16.233221. It reduces average inspection at the cost of accepting more bad lots in this example; it is not a universal improvement. ATI at AQL is 17.313145 and AOQ 0.009654, under the solver's rectifying convention. For double plans, QSB replaces n with ASN in the reported ATI and AOQ formulas. This is an aggregate convention: it does not separately calculate the inspected size conditional on each acceptance or rejection route.

The CLI can design the smallest *single* plan satisfying binomial bounds:

```
.build/debug/qsb design-acceptance 500 0.01 0.15 0.05 0.10
```

The result is $n = 25, c = 1$, producer's risk 0.025759, and consumer's risk 0.093071. Argument order is lot size, AQL, RQL, producer's risk, and consumer's risk. The command searches for a plan, rather than evaluating one already fixed. It does not design double plans, optimize expected cost, implement nonzero classification errors, or claim conformity with a specific sampling standard.

In macOS, the form distinguishes single/double plans and their decision thresholds; it preserves shared quality levels, risks, and economic data when the type changes. Validate and solve to see the OC curve, AQL/RQL markers, actual versus nominal risks, and ASN/ATI/AOQ. Economic fields are preserved in the model; core output contains quality and inspection measures, rather than an economic optimum. JSON allows the exact plan used to be audited.

15.8 Exercises

1. Add a count of twenty to the c example and observe how the center and limits change when it is included in estimation. Discuss when to establish a reference phase.

2. Compare p charts with sizes of one hundred and one thousand, keeping proportions fixed. Explain why the same deviation can produce different signals.

3. Design a single plan using the example's bounds and compare ASN with the $20, 1$ plan. Check actual risks, rather than only nominal ones.

Appendix A

Terminology reference

Backend Implementation selected to validate or solve a model family. The validation option does not compute a solution.

Balance Equality accounting for inflows, outflows, and changes in state, such as ending inventory equaling initial inventory plus production minus demand.

Capacity Resource limit, with a defined unit and time horizon.

Constraint Condition limiting the admissible decisions.

Decision variable Controllable quantity chosen by the decision maker in the model.

Diagnostic Validation message with severity, code, and, when available, the path of the affected field.

Exactness Characterization of the procedure and its guarantees for the supported model, distinct from the number of decimal places displayed.

Expected value Probability-weighted average; it does not guarantee that payoff in any particular realization.

Feasibility Satisfaction of the model's constraints, bounds, and conditions; it does not imply optimality.

Feasible region Set of solutions satisfying the constraints.

Global optimum Feasible solution with the best objective throughout the feasible region; a local optimum is compared only with nearby solutions.

Heuristic Procedure seeking a good solution without necessarily guaranteeing a global optimum.

Likelihood Probability of observing evidence conditional on a state; it is not the posterior probability of that state.

Linear relaxation Model replacing integrality requirements with continuous variables; its solution may be inadmissible for the integer problem.

Makespan Completion time of the last job in a schedule.

Normalized JSON Structured representation of a model or result that allows data to be exchanged, inspected, and reproduced across QSB clients.

Objective function Expression quantifying the criterion to maximize or minimize.

Optimal solution Feasible solution achieving the best objective of the problem considered, within the method's guarantees and tolerances.

Parameter Data item treated as known by the model during a run.

Policy Rule or set of decisions by stage and state.

Posterior Probability of a state after observing evidence, calculated from priors and likelihoods.

Seed Value initializing the pseudorandom generator; it enables a simulation to be repeated with the same model, configuration, and implementation.

Sensitivity Analysis of how the solution changes when data or assumptions change.

Shadow price Marginal change in the objective when a resource varies within the range and under the conditions where the dual interpretation is valid.

Slack Remaining capacity or margin of a constraint for a solution.

State Summary of sufficient information for future decisions in a dynamic model; it may also refer to a random condition of the environment.

Tolerance Numerical threshold used to compare quantities or accept approximations; it must be interpreted alongside the scale of the problem.

Trace Record of stages, states, and decisions helping to examine a computed policy; it is not necessarily the algorithm's entire internal table.

Utilization Fraction of service capacity occupied on average.

Value of information Expected improvement from allowing decisions based on evidence; EVSI refers to sample information and EVPI to perfect information.

Bibliography

[Ahuja et al., 1993] Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.

[Bertsimas and Tsitsiklis, 1997] Bertsimas, D. and Tsitsiklis, J. N. (1997). *Introduction to Linear Optimization*. Athena Scientific.

[Dantzig, 1963] Dantzig, G. B. (1963). *Linear Programming and Extensions*. Princeton University Press.

[Gross et al., 2008] Gross, D., Shortle, J. F., Thompson, J. M., and Harris, C. M. (2008). *Fundamentals of Queueing Theory*. Wiley, 4 edition.

[Harris, 1913] Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management*, 10:135–136.

[Hyndman and Athanasopoulos, 2021] Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, 3 edition.

[Law, 2015] Law, A. M. (2015). *Simulation Modeling and Analysis*. McGraw-Hill Education, 5 edition.

[Montgomery, 2020] Montgomery, D. C. (2020). *Introduction to Statistical Quality Control*. Wiley, 8 edition.

[Nemhauser and Wolsey, 1988] Nemhauser, G. L. and Wolsey, L. A. (1988). *Integer and Combinatorial Optimization*. Wiley.

[Pinedo, 2016] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition.

[Ross, 2019] Ross, S. M. (2019). *Introduction to Probability Models*. Academic Press, 12 edition.

[Silver et al., 1998] Silver, E. A., Pyke, D. F., and Peterson, R. (1998). *Inventory Management and Production Planning and Scheduling*. Wiley, 3 edition.

[Wolsey, 1998] Wolsey, L. A. (1998). *Integer Programming*. Wiley.